



Guía del administrador

Browser Policy Manager 0.9.3

Contenido

Requisitos del sistema y alcance de la entrega

- Requisitos mínimos del sistema para BPM
- Evaluar la preparación actual de la fuente de un solo nodo
- Registrar los límites diferidos de alta disponibilidad y producción

Implementar BPM desde código fuente en Linux

- Preparar un host Linux para la implementación de origen
- Configurar el pago del código fuente de Linux
- Configurar los ajustes del tiempo de ejecución de origen de Linux
- Verificar una implementación de origen de Linux
- Instalar BPM desde el código fuente en Ubuntu 26.04 LTS
- Instalar BPM desde el código fuente en Debian 13.5
- Instalar BPM desde el código fuente en Fedora Linux 44
- Instalar BPM desde el código fuente en Linux Mint 22.3
- Instalar BPM desde el código fuente en la rama estable de Manjaro

Implementar BPM desde código fuente con Windows WSL

- Preparar Windows 10/11 para la implementación desde el código fuente mediante WSL
- Configurar el control de origen BPM dentro de WSL
- Configurar el tiempo de ejecución de WSL y el acceso al navegador
- Verificar la implementación del origen BPM a través de WSL

Operar y actualizar BPM desde código fuente

- Revisar las fuentes de configuración de DevOps
- Planificar almacenamiento, registros y evidencia de respaldo
- Revisar la red, CORS y los límites de seguridad
- Registrar los límites operativos de la implementación desde código fuente
- Preparar evidencias antes de actualizar desde el código fuente
- Actualizar la revisión del código fuente y las dependencias
- Ejecutar migraciones y comprobaciones de actualización de origen
- Verificar la actualización y aplicar condiciones de detención de reversión
- Comprobar el estado y la preparación de BPM

Integrar BPM mediante la API

- Audiencia de integración y límites de la API
- Patrones de integración admitidos
- Convenciones API

- Limitaciones actuales de la API y de seguridad
- Reutilizar ejemplos de API y plantillas de entorno de DevOps
- Sincronizar datos del ciclo de vida del perfil
- Archivar, restaurar, eliminar permanentemente o restablecer perfiles
- Importar Firefox policies.json a través de la API
- Exportar un archivo Firefox policies.json canónico a través de la API
- Validar un candidato Firefox policies.json a través de la API

Ejecutar flujos de ciclo de vida y recuperarse de fallos

- Ejecutar un escenario de extracción, comparación y actualización de productos de control
- Ejecutar un escenario de importación, revisión, exportación y transferencia de cumplimiento
- Extraer y revisar perfiles BPM desde un producto de control externo
- Validar y aplicar una actualización de perfil con expected_revision
- Ejecutar transferencia de metadatos de importación, revisión, exportación y cumplimiento
- Iniciar el producto de control tras comprobar la integridad y la preparación de BPM
- Recuperarse de fallas de integración y registrar evidencia de auditoría

Resolver problemas y evaluar la preparación para producción

- Solucionar problemas de inicio fallido y sondas inalcanzables
- Solucionar problemas de caché de esquema y errores de validación de API
- Solucionar problemas de importación y exportación de Firefox
- Solucionar problemas de base de datos y almacenamiento
- Solucionar problemas de redes WSL y dependencias obsoletas
- Solucionar un portal de documentación no disponible
- Revisar las preguntas sobre exposición de la red y preparación de proxy
- Planificar entradas de monitoreo, evidencia de respaldo y ventanas de actualización

Asistente de documentación de BPM

- Operar el asistente de documentación en BPM 0.9.3
- Mantener el límite del asistente de documentación en BPM 0.9.3

Requisitos del sistema y alcance de la entrega

Requisitos mínimos del sistema para BPM

Use estos mínimos para el flujo de BPM desde código fuente compatible en la versión 0.9.3.

Plataforma base de BPM

Ejecute BPM mediante el flujo de código fuente Linux x86_64 compatible: Ubuntu 26.04 LTS, Debian 13.5, Fedora Linux 44, Linux Mint 22.3 o la rama estable actual de Manjaro. Windows 10 y Windows 11 solo se admiten como host de WSL 2 para ese flujo Linux. BPM no proporciona un servicio ni un instalador nativos de Windows. Use CPython 3.14, dos núcleos de CPU, 4 GiB de RAM y 4 GB de espacio libre escribible para el checkout, el entorno virtual, la base de datos, los registros y la compilación de documentación.

Navegador, red y acceso

Use un navegador de escritorio actual con JavaScript y HTTPS habilitados para la interfaz web de BPM y el portal de documentación. La lista de esquemas Firefox compatibles describe la compatibilidad de datos de políticas; no exige ejecutar la interfaz de BPM en Firefox. HTTPS saliente y sudo solo son necesarios al instalar paquetes del sistema o dependencias desde código fuente. El proceso necesita un usuario Linux normal que pueda escribir en el checkout de BPM y en sus directorios de datos configurados; no ejecute la aplicación como root solo para operarla.

Asistente de documentación en 0.9.3

El chat del asistente de documentación está disponible como interfaz del lector, pero la versión 0.9.3 no entrega un modelo local, un índice RAG, embeddings, respuestas generadas, citas ni acceso a fuentes externas. Por lo tanto, no añada requisitos de CPU, RAM, disco, GPU, NPU, cuenta en la nube ni red al flujo base de BPM.

Cada pregunta enviada muestra el aviso localizado de entrenamiento del modelo local. Use la búsqueda determinista independiente de la documentación para obtener información actual del producto.

Evaluar la preparación actual de la fuente de un solo nodo

Decida si la instancia BPM actual ejecutada en origen está lista para la operación controlada de un solo nodo y nombre la evidencia que respalda esa decisión.

Linux o Windows 10/11 a través de la implementación del código fuente WSL está completa, las dependencias están instaladas y se conocen los valores `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR` y `BPM_SCHEMA_CACHE_DIR` previstos.

BPM se puede preparar para la operación de fuente de un solo nodo en el estado actual. Esta evaluación no es un certificado de refuerzo de la producción, un diseño HA ni una aceptación de servicio empaquetado.

1. Registre la fuente exacta y la configuración del tiempo de ejecución.

Capture `git rev-parse --short HEAD`, la versión de BPM, `BPM_HOST`, `BPM_PORT`, `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR` y si el proceso se inicia con `make dev` o con un comando `uvicorn app.main:app` de propiedad directa.

2. Ejecute comprobaciones de actividad y preparación desde el mismo límite de red que utilizarán los usuarios.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Registre el resultado como evidencia `GET /health` y `GET /health/ready`. Las pruebas aprobadas sólo demuestran la vitalidad y preparación del proceso fuente.

3. Verifique los datos y la evidencia documental antes de compartir la instancia.

Confirme que `data/bpm.db` es la base de datos deseada, que al menos un perfil se puede exportar como `polices.json` y que `/help/` abre el artefacto de documentación compilado o un estado documentado no disponible.

4. Clasifica la instancia como operación de origen del estado actual.

Aviso:

No etiquete la instancia como empaquetada, reforzada, con carga equilibrada o de alta disponibilidad. Si los controles externos agregan esas propiedades, regístrelas como responsabilidades externas y mantenga la evidencia de BPM limitada a la operación de origen de un solo nodo.

La nota de preparación dice qué se puede hacer hoy de manera segura desde el código fuente, qué comprobaciones se aprobaron y qué decisiones deben esperar hasta la distribución y el soporte de tiempo de ejecución.

Utilice el tema exposición de la red y preparación del proxy antes de exponer la instancia más allá de una estación de trabajo de administrador local o un host de laboratorio controlado.

Registrar los límites diferidos de alta disponibilidad y producción

Indique qué capacidades de producción, HA, proxy inverso, restauración y gestión de servicios están ausentes en la actualidad y qué se puede preparar para trabajos de diseño posteriores.

Existen notas de preparación, exposición de la red, copia de seguridad/exportación, monitoreo y ventana de actualización para la instancia de ejecución de origen actual.

Este tema registra el límite entre la operación de origen de un solo nodo actual y la arquitectura de producción futura. Es deliberadamente explícito sobre lo que está planificado pero aún no implementado.

1. Nombre la forma actual admitida.

Escriba que la documentación del Administrador de BPM/DevOps admite la implementación de origen, la configuración manual, comprobaciones de estado/preparación, ejemplos de API, evidencia de copia de seguridad manual, comprobaciones de reconstrucción de documentación y ventanas de actualización de un solo nodo.

2. Nombre ausente de capacidades de servicio y proxy.

Enumere los servicios empaquetados, las unidades de servicio nativas de Linux, los servicios de Windows, las recetas oficiales de proxy inverso, las recetas de TLS/proxy, los secretos administrados, la configuración de la puerta de enlace de autenticación y el refuerzo de producción como ausentes en la documentación fuente de BPM.

3. Nombre ausente HA y capacidades de actualización.

Lista de agrupación HA admitida, coordinación de múltiples nodos, guía de afinidad de sesión del balanceador de carga, bloqueo de almacenamiento compartido, migración sin tiempo de inactividad, actualizaciones continuas, conmutación por error automática y automatización de restauración oficial como trabajo futuro, no como comportamiento actual.

4. Convertir las necesidades futuras en insumos de diseño.

Aviso:

No coloque varios nodos de origen detrás de un equilibrador de carga y llame al resultado admitido HA. Capture RTO/RPO requerido, verificación de respaldo, propiedad de TLS, propiedad de secretos, registro, monitoreo y expectativas de ventana de actualización como entradas para tareas posteriores de distribución/tiempo de ejecución.

El registro de límites evita la producción no compatible, HA y las reclamaciones de proxy inverso, al tiempo que preserva el trabajo de preparación útil para futuras versiones.

Utilice este registro durante la documentación manual QA y cuando se aprueben futuros elementos del instalador, servicio, proxy inverso y HA.

Implementar BPM desde código fuente en Linux

Preparar un host Linux para la implementación de origen

Confirme el límite de implementación de origen admitido, los requisitos previos del host y las capacidades de producción diferida antes de instalar BPM desde un repositorio.

Utilice un host Linux donde pueda instalar Python 3.14+, Git, herramientas de compilación y curl. Este runbook supone una extracción de código fuente, no un instalador empaquetado.

BPM se puede ejecutar desde el código fuente para desarrollo, revisión y evaluación administrativa controlada. Los instaladores empaquetados, las unidades de servicio nativas, las recetas oficiales de systemd, el proxy inverso y las recetas TLS, la agrupación en clústeres HA, los secretos administrados y el refuerzo de la producción no se entregan en esta versión y no deben presentarse como comportamientos de implementación admitidos.

1. Identifique el propósito de la implementación y registre que se trata de una implementación de Linux basada en código fuente.

Resultado esperado: la nota de operaciones dice que BPM se inicia desde un checkout de Git y que los instaladores específicos de la distribución se difieren.

2. Instalar o verificar los requisitos previos del host.

Las herramientas requeridas son Git, Python 3.14+, pip, make, curl y un shell que pueda ejecutar `source .venv/bin/activate`. Los comandos del repositorio utilizan el entorno virtual local y no requieren un paquete BPM global.

3. Cree un directorio de trabajo que esté separado de los secretos y los datos de producción.

Mantenga los datos de extracción del repositorio, `.venv` y SQLite local en una ruta controlada. No almacene secretos reales en ejemplos de documentación o archivos comprometidos.

4. Confirme lo que intencionalmente no está cubierto antes de continuar.

Aviso:

Este runbook no proporciona un instalador empaquetado, servicio nativo, unidad systemd compatible, receta de proxy inverso, terminación TLS, topología HA, almacén secreto administrado, automatización de respaldo ni lista de verificación de refuerzo de producción.

El host y los supuestos operativos están listos para un flujo de trabajo de extracción de origen BPM compatible.

Continúe con el tema de verificación de fuentes de Linux antes de configurar las variables de tiempo de ejecución.

Configurar el pago del código fuente de Linux

Clonar el repositorio BPM, crear el entorno virtual Python, instalar dependencias de desarrollo y preparar el esquema de la base de datos local.

El host Linux está preparado y el administrador tiene acceso a la URL del repositorio BPM y a la rama o etiqueta de destino para la versión 0.9.1.

El repositorio README define la secuencia de configuración de origen admitida. Utilice comandos locales del repositorio para que la resolución de problemas posterior pueda comparar el entorno con los scripts confirmados.

1. Clona el repositorio e ingresa al checkout.

```
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Reemplace los marcadores de posición con la URL del repositorio interno aprobado y la referencia de la versión.

2. Cree y active el entorno Python local.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

El entorno virtual permanece dentro del proceso de pago y no se confirma.

3. Instale las dependencias de origen de BPM desde la caja.

```
pip install -e ".[dev]"
```

La instalación editable mantiene el código de ejecución y las pruebas alineados con el árbol fuente actual.

4. Aplicar migraciones de bases de datos para la base de datos en tiempo de ejecución seleccionada.

```
alembic upgrade head
```

Aviso:

Ejecute migraciones solo en la base de datos BPM prevista. Para el flujo de trabajo de origen predeterminado, esta es la base de datos local SQLite, no una base de datos del servicio de producción.

La caja tiene un entorno Python activado, dependencias instaladas y un esquema de base de datos migrado.

Configure el entorno de ejecución antes de iniciar el proceso de solicitud.

Configurar los ajustes del tiempo de ejecución de origen de Linux

Establezca la URL de la base de datos, el host, el puerto, el modo de recarga, CORS y la ruta del sitio de documentación utilizada por el proceso BPM ejecutado en origen.

Las dependencias de pago están instaladas y se han aplicado las migraciones para la base de datos que planea usar.

BPM lee variables de entorno con el prefijo BPM_ y también admite un archivo .env local del repositorio. La URL de la base de datos predeterminada es `sqlite+aiosqlite:///./data/bpm.db`, que almacena datos en `data/bpm.db`.

1. Cree el directorio de datos local y defina la URL de la base de datos.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
```

Para ejecuciones de origen repetibles, el mismo valor puede almacenarse en un archivo .env local que no esté confirmado.

2. Establece valores de red y recarga para el proceso de origen local.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

Utilice loopback para verificación local. La vinculación a una interfaz compartida es una decisión operativa fuera de este límite del runbook de origen.

3. Revisar CORS y la configuración del sitio de documentación.

Los valores de origen predeterminados incluyen `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS` y `BPM_DOCUMENTATION_SITE_DIR`. Mantenga CORS permisivo solo para evaluación local controlada y apunte `BPM_DOCUMENTATION_SITE_DIR` a un sitio de documentación estática instalado cuando haya uno disponible.

4. Inicie BPM desde el script del repositorio.

```
make dev
```

El Makefile ejecuta `uvicorn app.main:app --reload --port 8000`. Mantenga abierta la terminal de primer plano para que los mensajes de inicio y los rastreos sean visibles.

BPM se ejecuta desde el origen con configuraciones de tiempo de ejecución local explícitas.

Verificar la salud, la preparación, el producto UI y las primeras comprobaciones administrativas de humo.

Verificar una implementación de origen de Linux

Utilice sondas de estado, la web UI, registros y un comando de prueba limitado para confirmar que la instancia de BPM ejecutada en origen es utilizable.

El proceso BPM se ejecuta desde el repositorio con la configuración prevista de BPM_DATABASE_URL, host, puerto y sitio de documentación.

La aplicación actual expone la actividad en /health y la preparación en /health/ready. Estas sondas confirman únicamente la respuesta del proceso en ejecución; no prueban el endurecimiento de la producción, la aplicación de Firefox, la preparación de copias de seguridad o el comportamiento HA.

1. Verifique la actividad y la preparación del host Linux.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Las respuestas esperadas son {"status": "ok"} y {"status": "ready", "ready": true}.

2. Abra el producto UI y confirme que la ruta de la Biblioteca responde.

Utilice `http://127.0.0.1:8000/profiles`. La ruta debe cargar la interfaz BPM o mostrar un estado intencional de la aplicación, no un rastreo del servidor.

3. Inspeccione los registros de la aplicación en primer plano.

Revise el terminal que ejecuta `make dev` o `uvicorn app.main:app --reload --port 8000`. Registre errores de inicio, errores de migración, conflictos de puertos y advertencias del sitio de documentación sin copiar secretos en tickets.

4. Registre el límite del resultado de verificación.

Aviso:

Superar las sondas de estado, la ruta de la interfaz y los registros en primer plano no certifica la preparación para producción, el comportamiento del proxy inverso, las copias de seguridad gestionadas ni la aplicación de políticas de Firefox en tiempo de ejecución.

El administrador tiene evidencia de que la implementación de origen de Linux se inicia, responde a las sondas de estado/preparación, sirve la ruta UI y no tiene bloqueadores de registros de primer plano inmediatos.

Si la verificación falla, mantenga juntos el comando, los nombres de las variables de entorno, las respuestas de la sonda y las líneas de registro relevantes para solucionar problemas. No amplíe este runbook a instrucciones de instalador diferido o de topología de producción.

Instalar BPM desde el código fuente en Ubuntu 26.04 LTS

Instalación completa de BPM 0.9.1 con Python 3.14 de la distribución.

Se requieren Ubuntu 26.04 LTS, sudo, HTTPS y una referencia Git aprobada.

Es una ejecución controlada de un nodo, no un servicio del sistema.

1. Compruebe Ubuntu e instale los requisitos.

```
. /etc/os-release
test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
python3.14-venv
export BPM_PYTHON=/usr/bin/python3.14
"$BPM_PYTHON" --version
```

2. Obtenga la referencia aprobada.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Instale BPM y migre la base.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Inicie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Compruebe sondas, interfaz y registros.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Deténgase con seguridad.

Use Ctrl+C, conserve la salida y no ejecute `alembic downgrade`.

La migración y la documentación finalizan y las sondas responden.

La secuencia de comandos se validó en un contenedor limpio de espacio de usuario de Ubuntu 26.04. Esta evidencia no valida el arranque nativo, el kernel, systemd, el escritorio, el hardware, el cortafuegos del host ni el despliegue en producción.

Instalar BPM desde el código fuente en Debian 13.5

Instalación completa con CPython 3.14.6 de usuario verificado.

Se requieren Debian 13.5, sudo, HTTPS, 4 GB libres y una referencia aprobada.

No se sustituye Python 3.13 del sistema; Python 3.14.6 se instala en \$HOME/.local.

1. Instale los requisitos de compilación.

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. Compile CPython 3.14.6 verificado.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Obtenga BPM.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Instale y migre BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```



```
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Inicie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Compruebe el resultado.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Deténgase con seguridad.

Conserve suma y registros; no rebaje la base ni elimine Python antes del diagnóstico.

Python, BPM, migración, documentación y sondas están preparados.

La secuencia de comandos se validó en un contenedor limpio de espacio de usuario de Debian 13.5. CPython indicó que los módulos opcionales `_dbm` y `_zstd` no estaban disponibles, pero el flujo completo de BPM se completó correctamente. Esta evidencia no valida el arranque nativo, el kernel, systemd, el escritorio, el hardware, el cortafuegos del host ni el despliegue en producción.

Instalar BPM desde el código fuente en Fedora Linux 44

Instalación completa con Python 3.14 de Fedora.

Se requieren Fedora 44, sudo, HTTPS y una referencia aprobada.

Es una ejecución de un nodo, no una receta systemd o de producción.

1. Instale los requisitos.

```
. /etc/os-release
test "$ID" = "fedora" && test "$VERSION_ID" = "44"
sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
export BPM_PYTHON=/usr/bin/python3
"$BPM_PYTHON" --version
```

2. Obtenga BPM.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Instale y migre BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Inicie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Compruebe el resultado.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Deténgase con seguridad.

Conserve salidas y registros y no rebaje la base.

BPM está migrado, la documentación generada y las sondas responden.

La secuencia de comandos se validó en un contenedor limpio de espacio de usuario de Fedora 44. Esta evidencia no valida el arranque nativo, el kernel, systemd, el escritorio, el hardware, el cortafuegos del host ni el despliegue en producción.

Instalar BPM desde el código fuente en Linux Mint 22.3

Ejecute la secuencia completa de instalación del código fuente de BPM 0.9.1 en Linux Mint 22.3 instalando un CPython 3.14.6 verificado con suma de verificación bajo el usuario actual.

Utilice Linux Mint 22.3 (Zena) con acceso sudo, HTTPS saliente, al menos 4 GB espacio de compilación libre y una referencia Git BPM 0.9.1 aprobada.

Linux Mint 22.3 utiliza la base de paquetes Ubuntu Noble, cuyo Python predeterminado está por debajo del requisito 3.14 de BPM. El sistema Python no se reemplaza.

1. Verifique Mint e instale los requisitos previos de compilación de CPython.

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. Compile la versión fuente verificada CPython 3.14.6.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Consulte la referencia de fuente BPM aprobada.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Cree el entorno, instale BPM y migre.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Inicia BPM en primer plano.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Verifique las sondas y el UI desde un segundo terminal.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Aplicar el límite de parada y reversión.

En caso de error, presione Ctrl+C, conserve los registros y la salida de la suma de comprobación y no ejecute alembic downgrade. Mantenga el prefijo de Python propiedad del usuario y realice el pago hasta el diagnóstico.

El host Linux Mint 22.3 tiene un tiempo de ejecución Python 3.14.6 propiedad del usuario, extracción de fuente de BPM migrada, documentación compilada, paso de sondas, UI de respuesta y registros de primer plano.

La secuencia de comandos se validó en un contenedor limpio de espacio de usuario de Linux Mint 22.3. CPython indicó que los módulos opcionales `_dbm` y `_zstd` no estaban disponibles, pero el flujo completo de BPM se completó correctamente. Esta evidencia no valida el arranque nativo, el kernel, systemd, el escritorio, el hardware, el cortafuegos del host ni el despliegue en producción.

Instalar BPM desde el código fuente en la rama estable de Manjaro

Instalación completa tras actualizar toda la rama estable y comprobar Python 3.14.

Use la rama estable posterior a 2026-06-26 con sudo, HTTPS y referencia aprobada.

Manjaro es continuo; registre rama y estado completamente actualizado.

1. Actualice e instale requisitos.

```
. /etc/os-release
test "$ID" = "manjaro"
test "$(pacman-mirrors -G)" = "stable"
sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
pacman-mirrors -G
uname -r
export BPM_PYTHON=/usr/bin/python
"$BPM_PYTHON" --version
```

2. Obtenga BPM.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Instale y migre BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Inicie BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Compruebe el resultado.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Deténgase con seguridad.

Conserve rama, paquetes y registros; no rebaje parcialmente paquetes ni base.

BPM y la documentación están listos y las sondas responden.

La secuencia de comandos se validó en un contenedor limpio de espacio de usuario de la rama estable de Manjaro después de registrar la rama y el estado completo de los paquetes. Esta evidencia no valida el arranque nativo, el kernel, systemd, el escritorio, el hardware, el cortafuegos del host ni el despliegue en producción.

Implementar BPM desde código fuente con Windows WSL

Preparar Windows 10/11 para la implementación desde el código fuente mediante WSL

Habilite WSL, elija una suposición de distribución de Linux compatible y confirme que BPM se ejecutará dentro de Linux en lugar de como un servicio nativo de Windows.

Utilice Windows 10 o Windows 11 con permiso para habilitar WSL 2, instale una distribución de Linux como Ubuntu LTS y ejecute comandos tanto en PowerShell como en el shell WSL.

Este runbook documenta la implementación del código fuente BPM a través de WSL. El proceso de solicitud admitido sigue siendo el flujo de trabajo fuente de Linux dentro de WSL; Los instaladores nativos de Windows, los servicios de Windows, los paquetes MSI, las tareas programadas, el alojamiento IIS y la administración de servicios de producción de Windows no se entregan en esta versión.

1. Confirme la disponibilidad de WSL de PowerShell.

```
wsl --status  
wsl --list --verbose
```

Resultado esperado: WSL está instalado, WSL 2 está disponible y la distribución de Linux elegida está visible.

2. Instale WSL y una distribución de Linux aprobada si faltan.

```
wsl --install  
wsl --install -d Ubuntu
```

Utilice el nombre de distribución aprobado por su organización. Reinicie si Windows lo solicita, luego cree el usuario de Linux cuando se inicie la distribución por primera vez.

3. Registre la distribución y el límite del sistema de archivos.

Coloque BPM bajo el sistema de archivos WSL Linux, por ejemplo /home/<linux-user>/browser-policy-manager. Evite ejecutar el pago desde /mnt/c porque la semántica del sistema de archivos de Windows y el escaneo antivirus pueden hacer que los entornos Python, la observación de archivos y los permisos no sean confiables.

4. Confirme lo que esta ruta de Windows aún no admite.

Aviso:

Esta no es una instalación nativa de Windows. El runbook no proporciona paquetes MSI/EXE, un servicio de Windows, un modelo de cuenta de servicio, integración del Visor de eventos, política de firewall de Windows, receta de proxy inverso/TLS, topología HA, secretos administrados, automatización de copias de seguridad ni refuerzo de producción.

Windows y WSL están listos para una implementación de fuente BPM basada en Linux con límites claros de plataforma y sistema de archivos.

Continúe abriendo el shell de distribución WSL y configurando el pago del código fuente en el directorio de inicio de Linux.

Configurar el control de origen BPM dentro de WSL

Instale los requisitos previos de Linux en WSL, clone BPM en el sistema de archivos de Linux, cree el entorno virtual, instale dependencias y aplique migraciones.

Se instala y abre una distribución de Linux WSL 2, y el administrador tiene la URL del repositorio BPM aprobada y la rama o etiqueta 0.9.1.

Todos los comandos de este tema se ejecutan dentro del shell de WSL Linux a menos que el paso diga explícitamente PowerShell. Esto mantiene la configuración alineada con el runbook de implementación de origen de Linux y evita la ambigüedad de la ruta nativa de Windows.

1. Instale los requisitos previos de Linux en la distribución WSL.

```
sudo apt-get update
sudo apt-get install -y git make curl python3 python3-venv python3-pip
python3 --version
```

Utilice el administrador de paquetes y los nombres de paquetes aprobados para su distribución WSL seleccionada.

2. Clonar BPM en el sistema de archivos doméstico de Linux.

```
cd ~
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

No coloque el pago en funcionamiento en /mnt/c para el flujo de trabajo de origen WSL compatible.

3. Cree y active el entorno Python local.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e "[dev]"
```

Si la distribución expone Python solo como python3, cree el entorno con `python3 -m venv .venv` y luego use el entorno activado.

4. Aplicar migraciones de bases de datos dentro del proceso de pago de WSL.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Aviso:

El archivo SQLite es un archivo de Linux dentro de WSL. No edite ni mueva `data/bpm.db` desde las herramientas de Windows mientras BPM se esté ejecutando.

El checkout de WSL tiene requisitos previos de Linux, un entorno virtual de repositorio local, dependencias instaladas y una base de datos local migrada.

Configure las variables de tiempo de ejecución y el acceso al navegador antes de iniciar BPM.

Configurar el tiempo de ejecución de WSL y el acceso al navegador

Configure las variables de tiempo de ejecución de BPM dentro de WSL, elija una dirección de enlace segura, inicie el proceso fuente e identifique la URL del navegador desde Windows.

El checkout de BPM, el entorno virtual, las dependencias y las migraciones están listos dentro del sistema de archivos WSL Linux.

BPM lee BPM_ variables de entorno del proceso de Linux. El WSL moderno normalmente reenvía localhost a Windows, pero el WSL IP se puede verificar cuando el acceso al navegador o el comportamiento del firewall no están claros.

1. Establece variables de tiempo de ejecución local en el shell WSL.

```
source .env/bin/activate
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
export BPM_HOST="0.0.0.0"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

BPM_HOST="0.0.0.0" permite que el navegador de Windows acceda al proceso WSL a través del reenvío de host local. Utilice 127.0.0.1 solo cuando el navegador también esté dentro de WSL o se haya verificado el reenvío.

2. Inicie BPM desde el proceso de pago de WSL.

```
make dev
```

El Makefile inicia `uvicorn app.main:app --reload --port 8000`. Mantenga la terminal WSL abierta para mensajes de inicio, rastreos y resultados del observador de archivos.

3. Identifique la URL del navegador de Windows.

```
hostname -I
```

Pruebe primero `http://localhost:8000/profiles` desde Windows. Si el reenvío de host local no está disponible, utilice el primer WSL IP informado por `hostname -I`, por ejemplo, `http://<wsl-ip>:8000/profiles`.

4. Mantenga explícitos los límites de seguridad.

Aviso:

Abrir el puerto WSL a otros hosts, cambiar las reglas del firewall de Windows, agregar TLS o colocar un proxy inverso delante de BPM está fuera de este tema de implementación de origen WSL y debe esperar la guía de preparación para la producción.

El proceso fuente BPM se ejecuta dentro de WSL y tiene una ruta de acceso al navegador de Windows identificada.

Verifique el estado, la preparación, el acceso UI y los modos de error específicos de WSL.

Verificar la implementación del origen BPM a través de WSL

Verifique las sondas de WSL y Windows, confirme la ruta UI, inspeccione los registros y solucione problemas comunes de redes y sistemas de archivos de WSL.

BPM se está ejecutando desde el proceso de pago de WSL con la configuración prevista de BPM_DATABASE_URL, host, puerto y sitio de documentación.

La verificación debe demostrar tanto la respuesta del proceso de Linux como la ruta de acceso del navegador de Windows. No certifica la instalación nativa de Windows, el refuerzo de la producción, el comportamiento del proxy inverso, la preparación de la copia de seguridad ni la aplicación del tiempo de ejecución de Firefox.

1. Verificar el estado y la preparación dentro de WSL.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Las respuestas esperadas son {"status":"ok"} y {"status":"ready","ready":true}.

2. Verifique las mismas sondas desde Windows PowerShell.

```
Invoke-WebRequest http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/health/ready
```

Si localhost no funciona, vuelva a intentarlo con el WSL IP de hostname -I. Registre qué dirección tuvo éxito.

3. Abra la ruta UI desde el navegador de Windows.

Utilice `http://localhost:8000/profiles` o `http://<wsl-ip>:8000/profiles`. La ruta de la Biblioteca debe cargar la interfaz BPM o un estado vacío/error intencional, no un rastreo.

4. Solucionar fallas específicas de WSL antes de cambiar el código BPM.

En caso de fallas de conexión, confirme que `make dev` aún se esté ejecutando, verifique BPM_HOST, ejecute `wsl --list --verbose` y actualice el WSL IP después de reiniciar la distribución. Para ver archivos lentamente o errores de permisos, mueva el proceso de pago de `/mnt/c` a `/home/<linux-user>`. Ejecute solo después de que la ruta del entorno y la ruta de la red sean estables.

Aviso:

Pasar las comprobaciones de estado a través de WSL no prueba la preparación para la producción, el comportamiento del servicio de Windows, las copias de seguridad administradas ni la compatibilidad con HA.

El administrador tiene evidencia de que BPM se inicia dentro de WSL, responde a sondeos, es accesible desde Windows y no tiene bloqueadores inmediatos de redes WSL ni de sistemas de archivos.

Mantenga la salida de PowerShell, los comandos de shell WSL, los nombres de variables de BPM_*, las respuestas de la sonda y los registros de primer plano juntos para solucionar problemas sin copiar secretos.

Operar y actualizar BPM desde código fuente

Revisar las fuentes de configuración de DevOps

Identifique las fuentes de configuración de BPM actuales, registre los valores de tiempo de ejecución efectivos y separe las configuraciones ordinarias de las expectativas de administración de secretos no compatibles.

BPM se implementa desde el código fuente en Linux o mediante WSL, y el administrador puede inspeccionar el entorno del shell, el archivo `.env` opcional y la verificación del código fuente.

BPM carga configuraciones de variables de entorno con el prefijo `BPM_` y desde un archivo raíz del repositorio `.env`. El modelo de configuración reside en los valores predeterminados documentados. Esta versión no define un almacén secreto de aplicaciones, una bóveda de credenciales, una configuración de token de API, una configuración de clave de cifrado ni un perfil de configuración de producción.

1. Enumere las fuentes de configuración utilizadas para esta ejecución.

Registre si los valores provienen de exportaciones de shell, de un `.env` local o de valores predeterminados en los valores predeterminados documentados. Mantenga `.env` local para el pago y no lo confirme.

2. Registre la configuración de tiempo de ejecución que afecta el comportamiento del proceso.

Incluye `BPM_APP_NAME`, `BPM_APP_VERSION`, `BPM_DEBUG`, `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_API_PREFIX`, `BPM_DEFAULT_LOCALE`, `BPM_SUPPORTED_LOCALES`, `BPM_I18N_DIR` y `BPM_DOCUMENTATION_SITE_DIR`. Los valores predeterminados son valores predeterminados de fuente local, no políticas de producción.

3. Registre los datos, el esquema y la configuración relacionada con la red por separado.

Seguimiento de `BPM_DATABASE_URL`, `BPM_DB_ECHO`, `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` y `BPM_SCHEMA_HTTP_TIMEOUT`. Trate la base de datos URL y los orígenes de la red como evidencia operativa incluso cuando no contengan contraseña.

4. Mantener secretos fuera de los ejemplos de configuración de BPM.

Aviso:

La documentación de implementación de origen de BPM no proporciona configuración de autenticación, secretos administrados, configuración de cifrado en reposo, rotación de tokens, aislamiento de inquilinos ni garantías de redacción de secretos. Si su entorno necesita esos controles, regístrelos como controles de plataforma externa en lugar de funciones compatibles con BPM.

La implementación tiene un inventario de configuración revisado que coincide con la configuración de origen actual y no implica un comportamiento de administración de secretos no compatible.

Utilice los temas de almacenamiento, red y límites operativos para revisar el impacto de los valores registrados antes de exponer BPM más allá de un entorno local controlado.

Planificar almacenamiento, registros y evidencia de respaldo

Localice la base de datos de implementación de origen, la caché de esquema, el sitio de documentación, los registros de primer plano y la evidencia de copia de seguridad manual sin reclamar soporte de restauración o copia de seguridad administrada.

El inventario de configuración de implementación incluye `BPM_DATABASE_URL`, `BPM_SCHEMA_CACHE_DIR` y `BPM_DOCUMENTATION_SITE_DIR`.

La implementación de origen predeterminada almacena datos de la aplicación en SQLite en `sqlite+aiosqlite:///./data/bpm.db`, correspondiente a `data/bpm.db` en la raíz del repositorio. Los archivos de caché de esquema se encuentran en `app/schemas/mozilla` de forma predeterminada y un artefacto de documentación instalado se lee desde `app/documentation/site` cuando está presente.

1. Identificar los archivos y directorios que mantienen el estado operativo.

Registre la ruta del repositorio, `data/`, `data/bpm.db`, `app/schemas/mozilla` y `app/documentation/site`. En WSL, mantenga estas rutas en el sistema de archivos de Linux en lugar de en `/mnt/c`.

2. Defina un paso de evidencia de respaldo manual antes de realizar trabajos riesgosos.

Detenga el proceso BPM en primer plano, copie `data/bpm.db` a una ubicación de copia de seguridad controlada y registre la marca de tiempo, la revisión del origen, la URL de la base de datos, el tamaño del archivo y la suma de verificación. BPM no proporciona respaldo automatizado, recuperación en un momento dado ni orquestación de restauración probada.

3. Capturar registros del proceso fuente actual.

Las ejecuciones de código fuente escriben mensajes de inicio, resultados de acceso de Uvicorn, rastreos, errores de migración, conflictos de puertos y advertencias del sitio de documentación en la terminal de primer plano, `stdout` o `stderr`. En esta versión no se documenta ningún directorio de registro de aplicaciones BPM dedicado, política de rotación de registros, solicitud ID, registro de auditoría o integración del Visor de eventos.

4. Empareje evidencia de copia de seguridad de archivos con evidencia de exportación cuando la transferencia de perfiles sea importante.

Utilice ejemplos de exportación de API existentes para exportar documentos relevantes de Firefox `policies.json` y almacenar el estado de respuesta, el perfil ID, la revisión, el canal de esquema y el hash del documento exportado. No trate la evidencia de exportación como una restauración completa de la base de datos.

El administrador sabe dónde se encuentra el estado actual de implementación del origen y qué evidencia manual se puede recopilar antes de realizar actualizaciones o solucionar problemas.

Antes de cambiar las revisiones de origen o los esquemas de bases de datos, utilice el futuro runbook de actualización desde el origen y deténgalo cuando la copia de seguridad o la evidencia de restauración estén incompletas.

Revisar la red, CORS y los límites de seguridad

Revise el host, el puerto, la recarga, los sondeos de estado, los valores predeterminados de CORS y los límites de seguridad antes de hacer accesible una instancia de BPM ejecutada en origen.

El proceso fuente se puede iniciar con `make dev` y el administrador puede cambiar `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_ENABLE_CORS` y `BPM_CORS_ALLOW_ORIGINS`.

Los valores predeterminados actuales están optimizados para la operación de fuente local: `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `BPM_RELOAD="true"`, `BPM_ENABLE_CORS="true"` y orígenes permisivos equivalentes a `["*"]`. Estos valores predeterminados no son una política estricta de exposición de la red.

1. Elija la dirección de enlace más estrecha para el entorno actual.

Utilice `127.0.0.1` para comprobaciones locales de Linux. Utilice `0.0.0.0` solo cuando el reenvío del navegador WSL o una red de laboratorio controlada lo requiera. Registre el motivo de cualquier vinculación sin bucle invertido.

2. Puerta de acceso operativo con sondas de estado y preparación.

Utilice `GET /health` y `GET /health/ready` antes de los ejemplos de API o UI comprobaciones de humo. Las sondas exitosas muestran únicamente la actividad/preparación del proceso; no prueban el estado de la copia de seguridad de la base de datos, la aplicación de Firefox, la autenticación, la autorización o la topología de producción.

3. Restringir CORS para cualquier evaluación compartida.

Para experimentos locales, la permisividad `BPM_CORS_ALLOW_ORIGINS` puede ser aceptable. Para un host compartido, establezca orígenes explícitos y registre quién es el propietario del origen del navegador. La documentación de BPM no define protecciones CORS, CSRF con credenciales, autenticación de API ni aislamiento de inquilinos.

4. Mantenga los controles de seguridad no compatibles fuera del reclamo de BPM.

Aviso:

Esta versión no incluye terminación de TLS, recetas de proxy inverso, política de firewall, limitación de velocidad, firma de solicitudes, registro de auditoría, configuración de SSO, OAuth/OIDC, control de acceso basado en roles, escaneo de vulnerabilidades ni conmutación por error HA. Documentelos como controles externos si su entorno los agrega.

La postura de la red BPM ejecutada en origen está documentada como una exposición local o de laboratorio controlada en lugar de una arquitectura de seguridad de producción.

Si las sondas fallan o la exposición no está clara, utilice los temas de verificación de implementación de origen antes de continuar con los ejemplos de integración de API.

Registrar los límites operativos de la implementación desde código fuente

Cree una nota de transferencia de operaciones que indique el usuario del tiempo de ejecución, el modelo de proceso, las comprobaciones admitidas y las capacidades de producción diferida.

Se completaron las revisiones de configuración, almacenamiento, copia de seguridad, red y CORS para la instancia ejecutada en origen. La documentación

La documentación de BPM Administrator/DevOps actualmente admite la operación de ejecución de código fuente para una evaluación controlada. Aún no proporciona instaladores de distribución, servicios nativos, demonización oficial, topología de producción o manuales operativos operativos completos para cada control empresarial.

1. Registre el usuario de tiempo de ejecución y el modelo de proceso.

Identifique el usuario shell propietario del pago, `.venv, data/` y el proceso en ejecución `make dev` o `uvicorn app.main:app --reload --port 8000`. No reclame una cuenta de servicio compatible, unidad `systemd`, servicio de Windows o política de reinicio a menos que su plataforma externa la proporcione y sea propietaria.

2. Registre las comprobaciones que se admiten hoy.

Incluye revisión de fuente, valores de configuración activos, estado de migración, `/health`, `/health/ready`, humo de ruta UI, estado de ejemplo de API, revisión de registros en primer plano y evidencia de respaldo manual. Vincule evidencia de API con ejemplos de API reutilizables en lugar de duplicar procedimientos de punto final.

3. Registrar capacidades diferidas como bloqueadores o responsabilidades externas.

Mencione los instaladores empaquetados, los servicios nativos de Linux o Windows, el proxy inverso y las recetas TLS, la agrupación en clústeres HA, los secretos administrados, la configuración de autenticación, la automatización de copias de seguridad, la orquestación de restauración, los paneles de monitoreo, la rotación de registros y el refuerzo de la producción, como no lo proporciona la documentación fuente de BPM.

4. Adjunte referencias de recuperación orientadas al usuario sin transferir la propiedad.

Cuando un operador necesita comprender los síntomas de UI, vincúlelo a la solución de problemas de cara al usuario, como la recuperación de la conexión del producto. Mantenga la implementación, la API, la copia de seguridad y la propiedad de la red en la Guía del administrador/DevOps.

La nota de transferencia establece qué puede admitir la implementación de fuente actual, qué evidencia lo demuestra y qué responsabilidades de producción quedan fuera de BPM.

Utilice esta nota límite como punto de partida para el runbook de actualización desde el origen y las tareas posteriores de documentación de producción/HA/proxy inverso.

Preparar evidencias antes de actualizar desde el código fuente

Recopile la revisión del origen, la configuración, la copia de seguridad de la base de datos, la evidencia de exportación del perfil y las condiciones de detención antes de mover BPM a una revisión del origen más reciente.

BPM se ejecuta desde un origen Linux o WSL y el administrador ha completado la configuración, el almacenamiento, la copia de seguridad y las revisiones de los límites operativos.

Un procedimiento de actualización desde el código fuente cambia el código, las dependencias, el esquema de la base de datos y las expectativas de salida de la documentación. BPM no proporciona reversión automática, garantías de degradación de la base de datos, copia de seguridad administrada ni herramientas de actualización empaquetadas. Trate la recopilación de evidencia como una puerta previa a la actualización requerida.

1. Detenga el proceso BPM en primer plano y registre el estado de origen actual.

```
git status --short
git rev-parse --short HEAD
```

Continúe solo cuando se comprenda el estado del árbol de trabajo y la revisión de la fuente actual se registre como referencia de reversión.

2. Registre la configuración efectiva y las rutas de tiempo de ejecución.

Capture los activos `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, `BPM_HOST`, `BPM_PORT` y si los valores provienen de exportaciones de shell o `.env`. No copie secretos en la nota de actualización.

3. Crear evidencia de respaldo de base de datos manual mientras BPM está detenido.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Registre la marca de tiempo, la revisión actual, la URL de la base de datos, el tamaño del archivo y la suma de verificación. Si la implementación no utiliza `data/bpm.db`, haga una copia de seguridad de la ruta resuelta desde `BPM_DATABASE_URL`.

4. Exporte evidencia de perfil crítica antes de cambiar el código.

Utilice los ejemplos de exportación de API existentes para guardar los documentos `policies.json` de Firefox necesarios y registrar el estado de respuesta, el perfil ID, la revisión, el canal de esquema y el hash del documento exportado.

Aviso:

Si falta la copia de seguridad de la base de datos o la evidencia de exportación, detenga la actualización. No reemplace la evidencia de respaldo faltante con la suposición de que la reversión de Git o alembic downgrade será segura.

La actualización tiene un conjunto de evidencia verificada previa al cambio y una referencia de fuente anterior conocida.

Proceda a actualizar la revisión del código fuente y las dependencias solo después de que el conjunto de pruebas esté completo y revisado.

Actualizar la revisión del código fuente y las dependencias

Mueva la compra a una revisión más reciente aprobada y reinstale el entorno Python sin ocultar dependencias o fallas en el árbol de trabajo.

El conjunto de pruebas previo a la actualización existe, se registra la revisión anterior y se conoce una rama, etiqueta o confirmación de destino aprobada.

Las actualizaciones del código fuente de BPM son operaciones en el entorno Git y Python realizadas en el proceso de pago existente. El procedimiento no instala una distribución empaquetada, un servicio nativo o un administrador de actualización de producción.

1. Obtenga la referencia de la fuente aprobada.

```
git fetch --tags
git status --short
```

Deténgase si los cambios locales son inesperados. Conserve los cambios propiedad del usuario o del sitio en lugar de sobrescribirlos.

2. Pasar a la revisión de fuente más reciente aprobada.

```
git checkout <approved-new-ref>
git rev-parse --short HEAD
```

Registre la revisión de destino y compárela con el ticket de cambio aprobado o la nota de la versión.

3. Actualiza el entorno Python local del repositorio.

```
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Utilice el .venv existente a menos que la nota de actualización requiera explícitamente reconstruirlo. Registre las fallas de dependencia antes de cambiar la base de datos.

4. Detenerse ante fallas de fuente o dependencia no resueltas.

Aviso:

Si falla el pago, la instalación de dependencias o la activación del entorno virtual, no ejecute las migraciones. Vuelva a la revisión anterior registrada solo si la base de datos no se ha modificado, luego reinstale el conjunto de dependencias anterior y vuelva a ejecutar las comprobaciones de humo previas a la actualización.

El entorno de pago y Python se encuentra en la revisión de destino aprobada o la actualización se detuvo antes de que se realizaran cambios en la base de datos.

Ejecute migraciones de bases de datos y comprobaciones de documentación solo después de que la actualización de la dependencia se realice correctamente.

Ejecutar migraciones y comprobaciones de actualización de origen

Aplice migraciones de base de datos y verifique el despliegue BPM actualizado desde el origen después de cambiar la revisión.

La extracción y las dependencias se actualizaron exitosamente y la evidencia de la copia de seguridad de la base de datos se recopiló mientras BPM estaba detenido.

Las actualizaciones pueden introducir migraciones de base de datos. BPM documenta la migración hacia adelante con `alembic upgrade head`; no se garantiza una degradación automática ni una reversión segura del esquema.

1. Aplicar migraciones de bases de datos a la base de datos deseada.

```
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Utilice el valor de implementación real para `BPM_DATABASE_URL`. Deténgase inmediatamente si el resultado de la migración no es claro o falla.

2. Inicie el proceso BPM actualizado y verifique su interfaz actual.

Compruebe el estado y la preparación, y después abra la ruta Biblioteca. Si la documentación es necesaria para el despliegue, confirme que `/help/` informa de un artefacto instalado listo y no de un estado ausente o incompatible.

3. Deténgase ante errores de migración o de verificación en tiempo de ejecución.

Aviso:

Después de ejecutar una migración, no suponga que basta con revertir el checkout de Git. No ejecute `alembic downgrade` salvo que una nota de versión revisada confirme expresamente su compatibilidad con el par exacto de revisiones. Si se requiere una reversión, use la copia de seguridad de la base de datos anterior a la actualización como base segura de recuperación.

La revisión de origen actualizada completó las migraciones y las verificaciones en tiempo de ejecución, o la actualización se detuvo en un punto de error designado.

Inicie BPM y ejecute el tema de verificación posterior a la actualización antes de devolver la instancia a los usuarios o integradores.

Verificar la actualización y aplicar condiciones de detención de reversión

Inicie el proceso BPM actualizado, verifique las sondas, UI, la evidencia de API y el estado de la documentación, luego decida si aceptar, detener o restaurar desde la copia de seguridad.

Se han completado las verificaciones de migraciones, fuentes de humo y reconstrucción de documentación o se ha registrado un punto de falla.

La verificación posterior a la actualización demuestra que la ejecución del código fuente actualizado se puede utilizar para el entorno controlado actual. No demuestra la preparación para la producción, el comportamiento del proxy inverso, HA, la copia de seguridad administrada ni la aplicación del tiempo de ejecución de Firefox.

1. Inicie el proceso fuente actualizado y verifique las sondas.

```
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Las respuestas esperadas siguen siendo {"status": "ok"} y {"status": "ready", "ready": true}.

2. Verifique el producto UI, la evidencia de API y el estado de la documentación.

Abra <http://127.0.0.1:8000/profiles>, ejecute los ejemplos de API reutilizables requeridos, confirme la evidencia exportada de `policies.json` cuando sea necesario y verifique que `/help/` no informe documentación obsoleta, incompatible o faltante para el artefacto instalado.

3. Acepte la actualización solo cuando se registren todas las comprobaciones requeridas.

Registre la revisión anterior, la revisión del objetivo, el resultado de la migración, el resultado de la fuente de humo, la validación de la documentación/el resultado de la compilación, las respuestas de salud/preparación, el resultado del humo de UI, la evidencia de API/exportación y cualquier control de producción diferido conocido.

4. Aplicar condiciones de reversión o detención sin inventar la recuperación automática.

Si ocurre una falla antes de las migraciones, regrese a la revisión fuente anterior y reinstale las dependencias. Si se produce una falla después de las migraciones, detenga BPM, restaure la copia de seguridad de la base de datos verificada previa a la actualización, regrese a la revisión de origen anterior, reinstale las dependencias y repita las comprobaciones de estado/UI. Si faltan pruebas de respaldo, la restauración no está verificada o los datos cambiaron después de la migración, deténgase y escale en lugar de afirmar que la reversión se realizó correctamente.

Aviso:

No marque la actualización como completada cuando los sondeos fallan, la documentación está obsoleta, las exportaciones difieren inesperadamente, las migraciones son inciertas o la evidencia de reversión está incompleta.

La actualización se acepta con evidencia completa, se detiene en un bloqueador conocido o se restaura desde la copia de seguridad previa a la actualización con prueba explícita.

Utilice la evidencia resultante en la nota de transferencia de operaciones y en la planificación futura de producción/HA/proxy inverso.

Comprobar el estado y la preparación de BPM

Utilice las comprobaciones de funcionamiento y disponibilidad actuales como una comprobación previa ligera del estado operativo antes de los flujos de trabajo de API.

Establecer `$BPM_BASE_URL`. Para la configuración del host, el inicio del servicio, los registros, las actualizaciones del origen y los detalles del sistema operativo, utilice la Guía del administrador de implementación del origen; Este tema de API solo documenta la comprobación de integración.

Esta tarea cubre API-HEALTH-001 y API-HEALTH-002. BPM expone sondas sin prefijo en `/health` y `/health/ready`. BPM no expone diagnósticos de dependencia a través de estas sondas; no tienen versiones y no reemplazan la política de incidentes o reintentos específicos del producto.

1. Llame a la comprobación de funcionamiento antes de iniciar una breve secuencia API.

Ejemplo de solicitud: `GET $BPM_BASE_URL/health`. Ejemplo de respuesta: `200 {"status":"ok"}`.

Ejemplo de error: actualmente no se declara ningún error de aplicación; Las fallas de transporte, puerta de enlace o proceso se manejan fuera de BPM.

2. Llame a la comprobación de disponibilidad antes de que se completen las operaciones que dependen del inicio de la aplicación.

Ejemplo de solicitud: `GET $BPM_BASE_URL/health/ready`. Ejemplo de respuesta: `200 {"status":"ready","ready":true}`. Ejemplo de error: actualmente no hay ningún cuerpo de estado degradado o detalle de dependencia expuesto.

3. Utilice la preparación como puerta, no como un inventario de dependencia completo.

Cuando `ready` es `true`, el producto integrador puede continuar con las llamadas de descubrimiento, validación, importación, exportación o ciclo de vida del perfil. No debe inferir detalles del estado de la base de datos, detalles de la caché del esquema, estado del portal de documentación, estado de la cola o topología de implementación a partir de esta respuesta.

4. Continuar sondeando y reintentando decisiones en el producto en integración.

BPM no declara un intervalo de sondeo, un encabezado de reintento posterior, un objetivo de nivel de servicio, un contrato de disyuntor, una solicitud ID ni un webhook para cambios de sonda. Almacene su propia marca de tiempo, estado observado, tiempo de espera y decisión de reintento.

5. Separe las comprobaciones de protocolo de enlace de API de la solución de problemas de implementación de origen.

Si no se puede acceder a las sondas o la respuesta difiere de los ejemplos, cambie a la Guía del administrador de implementación de origen para realizar verificaciones de procesos, entorno, bases de datos, registros y actualizaciones desde el origen en lugar de expandir esta tarea de integración de API en instrucciones de implementación.

6. Registre el resultado del comprobación previa del estado operativo con la acción API descendente.

Aviso:

Una respuesta exitosa de estado o preparación no autentica a las personas que llaman, no autoriza operaciones, reserva revisiones de perfiles, no garantiza el éxito posterior de la API ni demuestra la aplicación del tiempo de ejecución de Firefox. Trátelo como una pequeña señal de verificación previa.

El producto de integración tiene un comprobación previa del estado operativo BPM mínimo y actual para su actividad y preparación sin depender de un comportamiento de implementación no documentado.

Después de una verificación de preparación exitosa, continúe con el descubrimiento de OpenAPI o el tema de flujo de trabajo de API específico para operaciones de validación, importación/exportación u ciclo de vida del perfil.

Integrar BPM mediante la API

Audiencia de integración y límites de la API

La sección de integración de API de la Guía de administración/DevOps está destinada a herramientas externas que utilizan la API actual de BPM sin asumir garantías no compatibles.

Audiencia

Utilice esta guía cuando cree o revise un producto de configuración, cumplimiento, orquestación, inventario o control que intercambie datos de perfiles y políticas de Firefox con BPM. Los lectores principales son integradores de API, revisores de seguridad, ingenieros de lanzamiento que necesitan un contrato repetible para sincronización de perfiles, puertas de validación, importación/exportación de políticas, metadatos de cumplimiento, inventario y controles de estado.

Límite de evidencia

La guía se basa en el contrato OpenAPI generado, el inventario de API mantenido, la fuente de ruta/modelo y las pruebas de API enfocadas. Documenta la API que BPM ya expone en la versión 0.9.1. No autoriza nuevos puntos finales, modelos de seguridad, esquemas de datos, semántica de reintento ni promesas de compatibilidad.

Roles de consumidor admitidos

Los consumidores admitidos son roles genéricos: administrador de configuración, analizador de cumplimiento, puerta de revisión de políticas, recopilador de inventario de perfiles, orquestador de implementación y monitor operativo. Pueden llamar a las operaciones de servicio, perfil, importación/exportación de Firefox, validación y estado enumeradas en el inventario de API.

Reclamaciones no respaldadas

No describa un conector específico de producto, integración de mercado, integración certificada, asociación, agente administrado o flujo de trabajo de proveedor a menos que ese conector realmente se envíe. Las rutas HTML, los activos estáticos, el esquema generado interfaces y los asistentes de Python con prefijo de subrayado no son una integración estable de la máquina API.

Propiedad de los datos

BPM posee el modelo de perfil normalizado, el estado de validación, la revisión, el estado del ciclo de vida, el objeto de cumplimiento opaco y la representación de exportación canónica de Firefox. Los productos integrados son propietarios de su programación, autorización, seguimiento de auditoría,

política de reintento, registro de excepciones, decisión de implementación y cualquier verificación del tiempo de ejecución fuera de BPM.

Patrones de integración admitidos

BPM admite patrones API genéricos para perfiles, validación, documentos de Firefox, metadatos, inventario y flujos de trabajo de salud.

Sincronización de perfiles

Un producto de integración puede enumerar perfiles, leer un perfil mediante ID, crear un perfil normalizado, parchear campos de perfil mutables, archivar, restaurar, eliminar permanentemente o restablecer la biblioteca. La sincronización debe tratar el perfil ID y la revisión como estado propiedad de BPM y no debe asumir operaciones masivas, mutación de nombres ni reversión automática en múltiples solicitudes.

Puertas de validación

Una puerta de validación puede enviar un documento canónico candidato `policies.json` al punto final de validación admitido antes de que un flujo de trabajo de implementación lo acepte. La puerta debe inspeccionar el cuerpo de la respuesta así como el código de estado, porque la validación puede devolver un estado HTTP exitoso con `ok=false`.

Importación y exportación de políticas

Una integración puede importar Firefox `policies.json` como un perfil BPM y exportar un perfil BPM guardado nuevamente al JSON canónico de Firefox. No intercambie el modelo de perfil normalizado con la forma canónica del documento de Firefox; importar/exportar es el límite entre esas representaciones.

Transferencia de metadatos de cumplimiento

La API expone un objeto `compliance` opaco en las operaciones de creación, actualización e importación de perfiles. Las integraciones pueden pasar sus propios metadatos a través de BPM, pero BPM no publica un esquema versionado para ese objeto y no valida la semántica de cumplimiento externo.

Inventario y controles sanitarios

Los consumidores de estilo inventario pueden enumerar perfiles, solicitar estadísticas para el mismo contexto de filtro, leer perfiles individuales y llamar a puntos finales de servicio, actividad y preparación. Las respuestas de estado son intencionalmente compactas y no exponen diagnósticos por dependencia, estado de la documentación o preparación de la implementación más allá de los campos fijos actuales.

Patrones no admitidos

La guía no documenta la autenticación, la autorización, los límites de velocidad, las claves de idempotencia, la solicitud identificadores, los webhooks, la exportación masiva, la CRUD masiva, la paginación del cursor, los conectores de productos específicos, las asociaciones con proveedores ni la orquestación transaccional porque BPM no expone esas garantías.

Convenciones API

Utilice estas convenciones al leer contratos OpenAPI de BPM y crear solicitudes de integración.

Base URL y descubrimiento

Los ejemplos utilizan `$BPM_BASE_URL` como marcador de posición de implementación. Descubra el contrato actual de `/openapi.json` y, solo para inspección interactiva, el esquema FastAPI interfaces en `/docs` y `/redoc`. El futuro portal de documentación del producto está reservado para `/help/`, por lo tanto, no trate a `/docs` como la ruta de documentación DITA.

Tipos de contenido

La mayoría de las operaciones programáticas utilizan cuerpos de solicitud y respuesta JSON. La importación de Firefox acepta `application/json` y `multipart/form-data`; los tipos de medios no admitidos pueden devolver 415. La exportación de Firefox devuelve `application/json` y puede agregar `Content-Disposition` cuando se solicita la salida de descarga.

Identificadores y ciclo de vida

El perfil identificadores son identificadores de ruta enteros propiedad de BPM. Almacene los `id` y `revision` devueltos después de las operaciones de creación, importación, lectura y actualización. Los nombres de los perfiles son únicos y no se pueden cambiar a través de `ProfileUpdate`. Los valores del ciclo de vida están activos, archivados y todo; `include_deleted` es un indicador de compatibilidad que debe interpretarse junto con el ciclo de vida.

Modelos y canales de esquema

`ProfileCreate`, `ProfileUpdate` y `ProfileRead` son formas de perfil normalizadas de BPM. La importación/exportación de Firefox utiliza `policies.json` canónico con `policies` de nivel superior. La validación utiliza `ValidationRequest` y canales de esquema compatibles, como Firefox ESR y Release; un mapeo de políticas simple sigue siendo el comportamiento de compatibilidad, no la forma externa preferida.

Filtrado, clasificación y paginación

Las operaciones de estadísticas y lista de perfiles admiten búsqueda, esquema, estado de validación, ciclo de vida y filtros de inclusión-eliminados. La paginación de la lista tiene un desplazamiento/límite con el valor predeterminado `limit=50`, el máximo `200` y `offset=0`. La clasificación utiliza campos descritos actuales como `created_at`, `updated_at`, `name`, `schema_version` y `id`, con orden `asc` o `desc`.

Estado y manejo de errores

Utilice códigos de estado y forma del cuerpo juntos. 200, 201 y 204 son familias exitosas, pero la validación puede devolver 200 con `ok=false`. Los cuerpos de error usan FastAPI `detail` y pueden contener una cadena, una lista de validación o un objeto BPM con `message`, `error` o `issues`.

Limitaciones actuales de la API y de seguridad

BPM documenta claramente las limitaciones actuales para que las integraciones no infieran garantías que no estén expuestas.

Restricciones de seguridad

La API actual no tiene capa de autenticación o autorización. CORS tiene como valor predeterminado los orígenes * a menos que la configuración de implementación lo anule. La documentación no debe implicar autenticación de token API, OAuth, control de acceso basado en roles, aislamiento de inquilinos o comportamiento de autorización de producción.

Versiones y compatibilidad

La API no tiene prefijo de versión, política de compatibilidad, encabezado de obsolescencia ni versión de tipo de medio. Las integraciones deben fijar la versión de BPM con la que prueban y observar `/openapi.json`, el inventario de API y las notas de la versión en lugar de asumir una compatibilidad con versiones anteriores garantizada.

Simultaneidad, idempotencia y reintento

Solo PATCH admite simultaneidad optimista a través de `expected_revision` opcional. Crear, importar, eliminar, restaurar, restablecer y exportar no exponen ETags, encabezados de condiciones previas, claves de idempotencia ni contratos de reintento. Los consumidores deben diseñar su propio comportamiento seguro de reintento y recuperación de nombres duplicados.

Límites masivos y de transacciones

BPM no expone ningún punto final de perfil masivo CRUD, ni punto final de exportación masiva, ni paginación de cursor ni contrato de transacción multioperación. BPM no revierte automáticamente una integración fallida de varios pasos; el producto integrador posee registros de auditoría y secuenciación de recuperación.

Limitaciones de sobres de error

Las respuestas de error no utilizan un sobre con versión estable. Los estados 400, 404, 409, 415, 422 y 503 pueden tener diferentes formas de `detail`. Los consumidores deben clasificarse por operación, estado y forma corporal documentada.

Superficies excluidas y HTML

Las seis operaciones de OpenAPI web son rutas de producto HTML, no integración de máquina estable API. /i18n/{locale}.json, /favicon.ico, /static/*, el esquema generado interfaces y los asistentes de Python con prefijo de guión bajo están excluidos de los temas de operación de integración.

Límites de respuesta de salud

/health y /health/ready exponen únicamente campos de estado compactos. La preparación no informa diagnósticos por dependencia, detalles de la base de datos, estado del esquema, estado del portal de documentación ni aprobación de la implementación. Los monitores operativos deben tratarlos como puntos finales mínimos de protocolo de enlace.

Reutilizar ejemplos de API y plantillas de entorno de DevOps

Utilice patrones de solicitud independientes del lenguaje, plantillas de entorno, comandos curl copiables, muestras multiparte y aserciones compactas de Python para los flujos de trabajo actuales de la API de BPM.

Establezca `BPM_BASE_URL` para la instancia de BPM bajo prueba. No incluya credenciales, nombres de inquilinos, nombres de host personales, hosts privados o perfil de producción identificadores en ejemplos reutilizables; BPM actualmente no documenta la autenticación API.

Esta tarea cubre API-HEALTH-001, API-HEALTH-002, API-PROFILE-001, API-PROFILE-003, API-PROFILE-004, API-PROFILE-005, API-FF-001, API-VAL-001 y API-FF-002. Los ejemplos utilizan intencionalmente `$BPM_BASE_URL`, nombres sintéticos, `release-153`, aserciones de respuesta y dispositivos desechables identificadores en lugar de hosts o secretos específicos del entorno.

1. Cree una plantilla de entorno reutilizable antes de copiar solicitudes curl.

Ejemplo de solicitud: `export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"`.

Ejemplo de respuesta: el shell mantiene una URL base, un canal de esquema, un nombre de perfil sintético y una ruta de importación opcional para toda la ejecución. Ejemplo de error: el shell sale de la asignación cuando falta un valor requerido.

```
# .env.devops-example.template
BPM_BASE_URL=
BPM_SCHEMA_CHANNEL=release-153
BPM_PROFILE_NAME=docs-api-example
BPM_JSON_IMPORT_PATH=./policies.json

export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
```

2. Controla cada secuencia reutilizable con sondas de estado y preparación.

Ejemplo de solicitud: `curl -fsS "$BPM_BASE_URL/health"` y `curl -fsS "$BPM_BASE_URL/health/ready"`. Ejemplo de respuesta: `{"status": "ok"}` y `{"status": "ready", "ready": true}`. Ejemplo de error: curl sale de un valor distinto de cero para errores de transporte o respuestas distintas de 2xx cuando se usa `-f`.

```
curl -fsS "$BPM_BASE_URL/health"
curl -fsS "$BPM_BASE_URL/health/ready"
```

3. Reutilice fragmentos del ciclo de vida del perfil para crear, enumerar, leer y actualizar de forma protegida.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles/\$PROFILE_ID y PATCH \$BPM_BASE_URL/api/profiles/\$PROFILE_ID. Ejemplo de respuesta: 201 ProfileRead, 200 ProfileRead[], 200 ProfileRead y un revision incrementado. Ejemplos de error: 400, 404, 409 expected_revision obsoleto o 422.

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
"$BPM_BASE_URL/api/profiles")"
PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])'
<<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
-d '{"flags":{
  "DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":$PROFILE_REVISION}' \
"$BPM_BASE_URL/api/profiles/$PROFILE_ID"
```

4. Reutilización de validación e importación de fragmentos JSON con forma de carga útil explícita.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 y POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json. Ejemplo de respuesta: 200 {"ok":true,"profile":"release-153"} y 201 ProfileRead. Ejemplos de error: 400, 409, 415, 422 o 503.

```
curl -fsS -H 'Content-Type: application/json' \
-d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
"$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

5. Reutilice fragmentos de importación y exportación de varias partes para transferencia basada en archivos.

Ejemplo de solicitud: multipart/form-data carga en POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json seguido de GET \$BPM_BASE_URL/api/export/profiles/\$PROFILE_ID/firefox/policies.json?pretty=1. Ejemplo de respuesta: 201 ProfileRead para importar y 200 application/json que contiene el canónico Firefox policies para exportar. Ejemplo de error: 404 perfil faltante o 422 perfil no válido ID.

```
curl -fsS \
-F "name=docs-multipart-import-example" \
-F "schema_version=$BPM_SCHEMA_CHANNEL" \
-F "compliance={\"source\":\"docs-devops-template\"}" \
-F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

```
curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"
```

6. Utilice un asistente compacto de Python cuando la integración necesite afirmaciones de respuesta reutilizables.

Ejemplo de solicitud: `session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`. Ejemplo de respuesta: el asistente genera inmediatamente a menos que el estado y el cuerpo JSON coincidan con la afirmación. Ejemplo de error: el producto integrado debe registrar errores de aserción, excepciones de tiempo de espera, `expected_revision` obsoletos o errores de API estructurados.

```
import os
import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}
```

Aviso:

Mantener el estado de la respuesta, el perfil ID, la revisión, el canal del esquema, el hash del documento exportado y el resultado de la aserción en evidencia externa. No copie nombres de host reales, secretos, claves API, valores secretos o identificadores de producción en ejemplos de documentación.

El producto integrado tiene ejemplos de Python y curl reutilizables que se pueden copiar de forma segura, independientes de los hosts de implementación, sincronizados con el comportamiento actual de la API de BPM y vinculados a aserciones de respuesta explícitas.

Combine estos ejemplos con los manuales operativos de extracción/lista/lectura, validación antes de aplicar, importación-revisión-exportación y auditoría de fallas al crear flujos de trabajo de productos de control externo completos.

Sincronizar datos del ciclo de vida del perfil

Utilice lista de perfiles, estadísticas, lea, cree y actualice puntos finales sin asumir un comportamiento masivo o transaccional.

Establezca `$BPM_BASE_URL` y lea `/openapi.json` para las formas actuales `ProfileCreate`, `ProfileUpdate` y `ProfileRead`.

Esta tarea cubre API-PROFILE-001, API-PROFILE-002, API-PROFILE-003, API-PROFILE-004 y API-PROFILE-005. La tienda devolvió `id` y `revision`; No infieras la identidad sólo a partir del nombre.

1. Lista de perfiles con filtros antes de elegir un perfil para sincronizar.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Ejemplo de respuesta: `200 ProfileRead[]`. Ejemplo de error: 422 para una entrada de consulta escrita o por intervalos no válida.

2. Lea las estadísticas del perfil para el mismo contexto de filtro cuando necesite recuentos.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Ejemplo de respuesta: `200 {"filtered":1,"total":3}`. Ejemplo de error: 422 por entrada de consulta no válida.

3. Lea el perfil seleccionado por número entero ID e incluya registros archivados solo cuando sea necesario.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Ejemplo de respuesta: `200 ProfileRead` con `id`, `revision`, marcas de tiempo, ciclo de vida y estado de validación. Ejemplo de error: 404 para un perfil faltante o no visible; 422 para ruta o entrada de consulta no válida.

4. Cree un perfil BPM normalizado solo cuando la integración sea propietaria del nuevo nombre del perfil.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/profiles` con JSON `{"name":"corp-baseline","schema_version":"release-153","flags":{},"compliance":{}}`. Ejemplo de respuesta: `201 ProfileRead`. Ejemplos de error: 400 esquema/perfil no válido, 409 nombre duplicado o 422 solicitud o validación de política.

5. Actualice los campos mutables con `PATCH` y envíe `expected_revision` cuando necesite simultaneidad optimista.

Ejemplo de solicitud: `PATCH $BPM_BASE_URL/api/profiles/42` con JSON `{"description":"Reviewed by control plane","expected_revision":7}`. Ejemplo de respuesta: `200 ProfileRead` con revisión incrementada después de una actualización de material. Ejemplos de error: 400 esquema/perfil no válido, 404 perfil faltante o archivado, 409 `expected_revision` obsoleto o error de validación 422.

6. Registre el perfil sincronizado ID, la revisión, el estado de validación, el canal de esquema y cualquier metadato de cumplimiento opaco en el producto integrado.

Aviso:

BPM no proporciona una transacción multioperación. Si un paso posterior falla, el producto integrador posee la recuperación, los registros de auditoría y las decisiones de reintento.

La integración tiene una ruta de sincronización de perfil reproducible para operaciones de lista, estadísticas, lectura, creación y actualización.

Utilice la tarea destructiva del ciclo de vida antes de las operaciones de archivado, restauración, eliminación permanente o restablecimiento.

Archivar, restaurar, eliminar permanentemente o restablecer perfiles

Utilice puntos finales de ciclo de vida de perfil destructivo con registros explícitos de revisión y recuperación.

Confirme el perfil de destino ID, el estado actual del ciclo de vida y el registro de aprobación externo antes de llamar a puntos finales destructivos.

Esta tarea cubre API-PROFILE-006, API-PROFILE-007, API-PROFILE-008 y API-PROFILE-009. BPM no tiene token de confirmación masiva, ni punto final de deshacer para eliminación permanente ni reversión de transacciones entre operaciones.

1. Archive un perfil cuando necesite eliminarlo de los flujos de trabajo activos y al mismo tiempo conservarlo para su revisión.

Aviso:

Archive es un cambio destructivo en el ciclo de vida de los flujos de trabajo activos. Registre el motivo de aprobación y el perfil ID antes de la llamada.

Ejemplo de solicitud: DELETE \$BPM_BASE_URL/api/profiles/42. Ejemplo de respuesta: 204 sin cuerpo JSON. Ejemplos de error: 404 perfil faltante o 422 entrada de ruta no válida.

2. Restaure un perfil archivado solo después de verificar si el nombre y el propósito del perfil aún son válidos.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/profiles/42/restore. Ejemplo de respuesta: 200 ProfileRead. Ejemplos de error: el perfil 404 no se puede restaurar ni encontrar; 422 entrada de ruta no válida.

3. Eliminar permanentemente un perfil solo cuando la política de retención externa permita la eliminación.

Aviso:

La eliminación permanente elimina un perfil activo o archivado y no tiene un punto final de deshacer de BPM. Exporte o registre la evidencia requerida antes de la llamada.

Ejemplo de solicitud: DELETE \$BPM_BASE_URL/api/profiles/42/hard. Ejemplo de respuesta: 204 sin cuerpo JSON. Ejemplos de error: 404 perfil faltante o 422 entrada de ruta no válida.

4. Restablezca la biblioteca solo en entornos desechables o explícitamente aprobados.

Aviso:

Restablecer elimina permanentemente todos los perfiles de la biblioteca y actualmente no requiere ningún token de confirmación. Nunca lo utilices como atajo de sincronización de rutina.

Ejemplo de solicitud: DELETE \$BPM_BASE_URL/api/profiles/reset. Ejemplo de respuesta: 200 {"deleted":3}. Comportamiento de error: actualmente no se declara ningún error de aplicación; Las fallas de la base de datos utilizan el manejo del marco.

5. Después de cualquier llamada destructiva, vuelva a enumerar los perfiles y lea el registro afectado cuando corresponda.

Ejemplo de solicitud: GET \$BPM_BASE_URL/api/profiles?lifecycle=all. Ejemplo de respuesta: 200 ProfileRead[]. Ejemplo de error: 422 entrada de consulta no válida. Utilice esta verificación para actualizar el estado del inventario del producto integrado.

6. Escriba el registro de auditoría externa con la operación ID, método HTTP, ruta, perfil ID, aprobador, motivo, marca de tiempo, estado de respuesta y decisión de recuperación.

Resultado esperado: un revisor posterior puede distinguir las acciones de archivar, restaurar, eliminar por completo y restablecer y puede ver por qué la integración consideró segura la operación.

Las operaciones destructivas del ciclo de vida del perfil están documentadas con evidencia de solicitud, respuesta, error, advertencia y recuperación.

Si falta el registro de aprobación, detenga el flujo de trabajo de integración antes de realizar una eliminación permanente o un restablecimiento.

Importar Firefox policies.json a través de la API

Cree un perfil BPM normalizado a partir de un documento canónico de Firefox Enterprise policies.json utilizando JSON o entrada multiparte.

Configure \$BPM_BASE_URL, elija un canal de esquema compatible como release-153 y prepare un documento completo de Firefox con un objeto policies de nivel superior.

Esta tarea cubre API-FF-001. La importación acepta application/json con FirefoxPoliciesJsonImportRequest o multipart/form-data con un campo file. BPM valida el documento canónico de Firefox, normaliza policies en el perfil flags y almacena metadatos opacos opcionales compliance en el perfil de BPM.

1. Envíe una solicitud de importación JSON cuando el producto integrador ya tenga el documento en la memoria.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json con Content-Type: application/json y JSON {"name": "api-json-import", "description": "Imported by control product", "schema_version": "release-153", "compliance": {"framework": "cis", "layer": "cis_l1"}, "document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}.

2. Utilice la importación multiparte cuando el sistema fuente entregue un archivo.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json con multipart/form-data, campos name=api-multipart-import, schema_version=release-153, description opcional, compliance={"framework": "cis", "layer": "cis_l2"} opcional como cadena JSON y file=@policies.json; type=application/json. Runtime también acepta un campo multiparte document, pero el esquema multiparte de OpenAPI actualmente declara file; utilice file para nuevos ejemplos.

3. Lea el perfil BPM creado a partir de la respuesta 201 ProfileRead.

Ejemplo de respuesta: 201 ProfileRead con schema_version="release-153", flags.DisableTelemetry=true, flags.BlockAboutConfig=true, almacenado compliance, id y revision. Almacene el perfil devuelto ID en lugar de inferir la identidad del nombre del perfil.

4. Trate los errores de importación como errores de creación de perfiles y no asuma un estado de perfil parcial.

Ejemplos de error: 400 para JSON con formato incorrecto, forma de documento incorrecta o canal de esquema no compatible; 409 para nombre de perfil duplicado; 415 para tipos de medios no compatibles; 422 para errores de validación de políticas de Firefox o solicitudes.

5. Mantenga las formas canónicas de Firefox y BPM normalizadas separadas en el código de integración.

La solicitud document.policies es el límite de implementación de Firefox. Los flags del perfil creado son el modelo normalizado de BPM. No envíe un objeto flags de nivel superior a este punto final de importación.

6. Registrar evidencia de importación en el producto integrante.

Aviso:

La importación crea un perfil BPM pero no implementa la configuración de Firefox, no demuestra la aplicación del tiempo de ejecución ni proporciona una clave de idempotencia. Registre externamente el hash del documento fuente, el canal del esquema, el perfil devuelto ID, la revisión, los metadatos de cumplimiento y la decisión de recuperación de nombres duplicados.

El documento `policies.json` de Firefox se representa como un perfil BPM normalizado que puede ser revisado, validado, exportado y rastreado por ID.

Utilice la tarea de exportación para devolver el perfil guardado al JSON canónico de Firefox antes de entregarlo a las herramientas de implementación.

Exportar un archivo Firefox policies.json canónico a través de la API

Represente un perfil BPM como documento de implementación de Firefox Enterprise con descarga opcional y parámetros de impresión bonita.

Almacena el perfil BPM ID devuelto por creación o importación. Si el perfil está archivado, decida explícitamente si la integración puede exportarlo con `include_deleted=true`.

Esta tarea cubre API-FF-002. Export lee el flags normalizado de BPM y devuelve el JSON canónico de Firefox con un objeto policies de nivel superior. Los metadatos del perfil BPM y los metadatos opacos compliance no se exportan al documento de implementación de Firefox.

1. Exporta el perfil activo como JSON canónico de Firefox.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`.

Ejemplo de respuesta: 200 application/json con JSON `{"policies":`

`{"DisableTelemetry":true,"BlockAboutConfig":true}}`.

2. Agregue el modo de descarga cuando la persona que llama necesita un nombre de archivo adjunto.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1`. Ejemplo de respuesta: 200 application/json más Content-Disposition:

`attachment; filename="profile-42-policies.json"`. El cuerpo JSON es el mismo documento de implementación.

3. Elija los parámetros de formato deliberadamente.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` selecciona la sangría de dos espacios cuando se omite indent. Ejemplo de solicitud: `GET`

`$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?indent=0` utiliza el parámetro de sangría explícito. Ejemplo de error: 422 para valores de consulta restringidos no válidos.

4. Exporte un perfil archivado solo cuando el flujo de trabajo externo tenga un motivo de revisión.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?include_deleted=true&download=1&pretty=1`. Ejemplo de respuesta: 200 application/json.

Ejemplos de error: 404 cuando un perfil faltante o archivado no es visible; 422 para ruta o entrada de consulta no válida.

5. Validar el documento exportado antes de entregarlo a otro sistema.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/validate/release-153` con JSON `{"document":`

`{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}`. Ejemplo de respuesta: 200 `{"ok":true,"profile":"release-153"}`. La validación confirma la forma del esquema, no el éxito de la implementación del tiempo de ejecución de Firefox.

6. Registre externamente el artefacto exportado exacto.

Aviso:

La exportación no tiene ETag, encabezado de condición previa, punto final masivo ni contenedor de transacción. Registre el perfil ID, la revisión observada antes de la exportación, el canal del esquema, los parámetros de consulta, el hash del documento exportado y la decisión de transferencia posterior fuera de BPM.

La integración tiene un artefacto canónico de Firefox policies.json que excluye los metadatos solo de BPM y se puede suministrar a las herramientas de implementación.

Si la validación falla o la revisión del perfil cambió entre lectura y exportación, detenga la transferencia y actualice el estado del perfil.

Validar un candidato Firefox policies.json a través de la API

Utilice el punto final de validación como puerta de verificación previa para los documentos candidatos de Firefox Enterprise policies.json y el manejo estructurado de errores.

Elija el canal de esquema de destino, como release-153 o esr-140.13, y prepare un cuerpo ValidationRequest con un documento completo de Firefox en document.policies.

Esta tarea cubre API-VAL-001. BPM valida el contenido candidato policies.json antes de la importación, la transferencia de exportación o una puerta de revisión de políticas externa. Aún se acepta una asignación de políticas simple por motivos de compatibilidad, pero los nuevos ejemplos de integración utilizan la forma completa de Firefox policies.json.

1. Envíe una solicitud de validación exitosa para el canal de esquema seleccionado.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 con JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Ejemplo de respuesta: 200 {"ok":true,"profile":"release-153"}.

2. Trate ok=false como una puerta fallida incluso cuando el estado de HTTP sea 200.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 con JSON {"document":123}. Ejemplo de respuesta: 200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}. Verifique el cuerpo de la respuesta, no solo el estado HTTP.

3. Maneja errores de documentos canónicos de Firefox con formato incorrecto como fallas en la forma de la solicitud.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 con JSON {"document":{"policies":[]}}. Ejemplo de error: 400 con detail.message="Firefox policies.json validation failed" y detail.issues[].path apuntando a policies.

4. Rechace canales de esquema no admitidos antes de confiar en los resultados de la validación.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/beta-999 con JSON {"document":{"policies":{"DisableTelemetry":true}}}. Ejemplo de error: 404 con texto detallado Unknown profile 'beta-999'. Un esquema registrado pero no disponible devuelve 503.

5. Asigne errores de validación de políticas a comentarios procesables de los revisores.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 con JSON {"document":{"policies":{"Proxy":{"Mode":"bogus"}}}}. Ejemplo de error: 422 con detail.message="Policy validation failed" y campos de problema policy, path y message.

6. Registre la decisión de validación en el producto integrador.

Aviso:

La validación es solo una puerta de esquema y forma de política. No implementa Firefox, no demuestra la aplicación del tiempo de ejecución, no autentica a las personas que llaman, no reserva una revisión de perfil ni proporciona una clave de idempotencia. Almacene el canal, el hash del documento candidato, el estado de la respuesta, el valor ok, los problemas estructurados y la decisión del revisor de forma externa.

La integración puede bloquear transferencias de candidatos `policies.json` son inseguras y preservar evidencia estructurada para decisiones de éxito y fracaso.

Si la validación se realiza correctamente, continúe importando, actualizando el perfil o exportando la transferencia utilizando el mismo canal de esquema y hash candidato registrado.

Ejecutar flujos de ciclo de vida y recuperarse de fallos

Ejecutar un escenario de extracción, comparación y actualización de productos de control

Utilice el perfil y la validación existentes API para extraer el estado de BPM, compararlo externamente y actualizar un perfil con propiedad explícita y evidencia de simultaneidad.

Establezca `$BPM_BASE_URL` y decida qué producto de control externo posee la lógica de comparación, el cálculo del estado deseado, la política de reintento y los registros de auditoría. BPM no expone una API de comparación ni un contenedor de transacciones.

Este escenario utiliza API-PROFILE-001, API-PROFILE-003, API-PROFILE-005 y API-VAL-001. BPM posee el estado del perfil almacenado y la validación. El producto integrado posee el estado deseado, el resultado de la comparación, la aprobación del revisor y las decisiones de reintento.

1. Extraiga perfiles de candidatos de la biblioteca con las mismas reglas de filtro que utiliza el producto de control para la propiedad.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0`. Ejemplo de respuesta: `200 ProfileRead[]`.
Ejemplo de error: 422 por entrada de consulta no válida.

2. Lee cada perfil seleccionado por ID y almacena su revision actual.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/42`. Ejemplo de respuesta: `200 ProfileRead` con `flags`, `compliance`, `schema_version` y `revision`. Ejemplos de error: 404 perfil faltante o archivado; 422 entrada de ruta no válida.

3. Compara el perfil extraído con el estado externo deseado fuera de BPM.

Resultado esperado: los registros del producto integrado cambiaron las claves de política, las claves sin cambios, las claves faltantes, el revisor y la fuente del estado deseado. No asuma que existe un punto final de comparación de BPM, un conector de mercado o un flujo de trabajo certificado.

4. Valide el documento candidato de Firefox antes de aplicar la actualización.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/validate/release-153` con JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Ejemplo de respuesta: `200 {"ok":true,"profile":"release-153"}`. Ejemplos de error: 400, 404, 422 o 503 según la forma del documento, el canal, la validez de la política y la disponibilidad del esquema.

5. Parchear el perfil BPM solo con la revisión observada antes de la comparación.

Ejemplo de solicitud: `PATCH $BPM_BASE_URL/api/profiles/42` con JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Ejemplo de respuesta: `200 ProfileRead` con `revision` incrementado. Ejemplos de error: 400, 404, 409 obsoleto `expected_revision` o 422.

6. Maneja fallas sin repetición automática.

Aviso:

Si la validación falla, la revisión está obsoleta o el resultado de la actualización no coincide con el estado deseado, detenga el flujo de trabajo, vuelva a leer el estado de BPM y cree un registro de auditoría externo. BPM no proporciona transacciones de operaciones múltiples, reversión automática, clave de idempotencia ni contrato de reintento seguro.

El producto de control tiene un flujo de extracción/comparación/actualización ejecutable con propiedad estatal explícita, evidencia de validación, protección de revisión y límites de recuperación de fallas.

Si el perfil actualizado debe implementarse en otro lugar, continúe con la tarea de exportación de Firefox y conserve el perfil ID, la revisión, la respuesta de validación y el registro de aprobación.

Ejecutar un escenario de importación, revisión, exportación y transferencia de cumplimiento

Utilice el estado, la importación, la validación, el perfil y la exportación existentes API para entregar un documento de Firefox revisado con evidencia de cumplimiento.

Establezca \$BPM_BASE_URL, elija el canal del esquema y decida qué sistema posee el documento fuente, el resultado del análisis de cumplimiento, la aprobación del revisor, el artefacto exportado y la decisión de recuperación.

Este escenario utiliza API-HEALTH-001, API-HEALTH-002, API-FF-001, API-PROFILE-003, API-VAL-001 y API-FF-002. BPM almacena el perfil normalizado y los metadatos de cumplimiento opacos; el producto integrador posee la interpretación del escaneo y la implementación posterior.

1. Comience con salud y preparación antes de la secuencia de transferencia.

Ejemplos de solicitud: GET \$BPM_BASE_URL/health y GET \$BPM_BASE_URL/health/ready. Ejemplos de respuesta: 200 {"status":"ok"} y 200 {"status":"ready","ready":true}. Ejemplo de error: el error de transporte o puerta de enlace se maneja fuera de BPM porque actualmente no se declara ningún error de aplicación.

2. Importe el documento de Firefox revisado con metadatos de cumplimiento adjuntos al perfil de BPM.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json con JSON {"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","result":"passed","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Ejemplo de respuesta: 201 ProfileRead. Ejemplos de error: 400, 409, 415 o 422.

3. Lea el perfil creado y registre los campos de propiedad de BPM por separado de la evidencia externa.

Ejemplo de solicitud: GET \$BPM_BASE_URL/api/profiles/42. Ejemplo de respuesta: 200 ProfileRead con id, revision, flags, schema_version y opaco compliance. Ejemplos de error: 404 o 422.

4. Valide el documento canónico de Firefox que se entregará.

Ejemplo de solicitud: POST \$BPM_BASE_URL/api/validate/release-153 con JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Ejemplo de respuesta: 200 {"ok":true,"profile":"release-153"}. Ejemplos de error: 400, 404, 422 o 503.

5. Exporte el perfil BPM como documento de implementación canónico de Firefox.

Ejemplo de solicitud: GET \$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1. Ejemplo de respuesta: 200 application/json con Content-Disposition y policies de nivel superior. Los metadatos del perfil BPM y los metadatos de cumplimiento no se exportan.

6. Deténgase en caso de discrepancias y escriba el registro de recuperación de fallas.

Aviso:

Si la evidencia de importación, validación, relectura o exportación difiere del documento fuente aprobado, no vuelva a intentarlo a ciegas. Almacene el hash de origen, el perfil ID, la revisión, el cuerpo de validación, el hash de exportación, el resultado del escáner, el revisor, el estado de respuesta y la decisión de recuperación fuera de BPM.

El producto integrado tiene un flujo ejecutable de importación, revisión y exportación con transferencia de cumplimiento, exportación canónica y evidencia de recuperación explícita.

Utilice herramientas de implementación fuera de BPM para aplicar el `policies.json` exportado; La validación de BPM no prueba la aplicación del tiempo de ejecución de Firefox.

Extraer y revisar perfiles BPM desde un producto de control externo

Cree el lado de extracción/lista/lectura de una integración de control-producto con propiedad explícita, filtros y límites de evidencia.

Establecer `$BPM_BASE_URL`. Decidir qué producto de control externo posee la selección de inventario, el almacenamiento de comparación, la política de reintento y la evidencia de auditoría; BPM posee únicamente los registros de perfil devueltos por su API.

Este flujo de trabajo utiliza API-PROFILE-001, API-PROFILE-002 y API-PROFILE-003. Es el punto de entrada seguro de solo lectura antes de los flujos de trabajo de comparación/actualización, validación antes de aplicar, importación-revisión-exportación o transferencia de metadatos de cumplimiento.

1. Confirme el alcance del producto de control antes de leer el estado de BPM.

Registre el propietario externo, el propósito, el canal del esquema, las reglas de filtrado y si los perfiles archivados están excluidos. Utilice `lifecycle=active` a menos que un procedimiento de recuperación necesite explícitamente `include_deleted=true`.

2. Listar perfiles con filtros deterministas y paginación.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Ejemplo de respuesta: 200 ProfileRead[]. Ejemplo de error: 422 para parámetros de consulta no válidos.

3. Capture recuentos de inventario opcionales por separado de los datos del perfil.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`.
Ejemplo de respuesta: 200 application/json. Almacene la evidencia de recuento como contexto de inventario de asesoramiento, no como un bloqueo o límite de transacción.

4. Lea cada perfil seleccionado por ID antes de la comparación.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Ejemplo de respuesta: 200 ProfileRead con id, revision, schema_version, flags y compliance. Ejemplos de error: 404 o 422.

5. Conservar una instantánea de lectura fuera de BPM para el producto de control.

Perfil de tienda ID, revisión, canal de esquema, consulta de filtro, marca de tiempo de lectura, URL de origen, claves de política seleccionadas y ejecución del producto de control ID. No almacene secretos ni nombres de host privados en ejemplos de documentación compartida.

6. Deténgase si no se puede confiar en el resultado de la extracción.

Aviso:

Si la paginación, el filtrado o la lectura de la evidencia están incompletos, deténgase antes de la actualización. BPM no expone la paginación del cursor, el aislamiento de instantáneas, los bloqueos de comparación, los webhooks ni un contrato de reproducción segura para los flujos de trabajo de lectura.

El producto de control externo tiene una instantánea de extracción/lista/lectura reproducible y sabe qué evidencia pertenece a BPM y qué evidencia pertenece al sistema externo.

Continúe con validar antes de aplicar o comparar/actualizar solo después de que el sistema externo haya registrado el perfil seleccionado identificadores y las revisiones.

Validar y aplicar una actualización de perfil con `expected_revision`

Ejecute un flujo de trabajo de actualización de validación primero que protege el estado de BPM con la revisión actual y se detiene ante datos obsoletos o no válidos.

El producto externo ya leyó el perfil de destino y almacenó su `revision` actual. Prepare el documento candidato de Firefox `policies.json` antes de enviar cualquier parche de perfil.

Este flujo de trabajo utiliza `API-VAL-001`, `API-PROFILE-003` y `API-PROFILE-005`. Documenta la ruta de actualización optimista actual; BPM no proporciona ETags, encabezados de condiciones previas, claves de idempotencia ni transacciones de operaciones múltiples.

1. Cree el documento candidato de Firefox fuera de BPM.

Registre las claves de política deseadas, el sistema de origen, el revisor, el ticket y el canal de esquema. Candidato de ejemplo: `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Validar el candidato antes de parchear el perfil.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/validate/release-153`. Ejemplo de respuesta: `200 {"ok":true,"profile":"release-153"}`. Ejemplos de error: 400, 404, 422 o 503.

3. Vuelva a leer el perfil inmediatamente antes de aplicar la actualización cuando la ventana de aprobación sea larga.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/42`. Ejemplo de respuesta: `200 ProfileRead`. Si `revision` cambió, vuelva a ejecutar la comparación y aprobación en lugar de parchear el estado obsoleto.

4. Parchear solo campos mutables con la revisión observada.

Ejemplo de solicitud: `PATCH $BPM_BASE_URL/api/profiles/42` con JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Ejemplo de respuesta: `200 ProfileRead` con `revision` incrementado.

5. Trate las respuestas conflictivas como señales de alto.

Ejemplos de error: 400 carga útil no válida, 404 perfil faltante, 409 `expected_revision` obsoleto y 422 errores de solicitud a nivel de esquema. Un 409 significa que se debe recalcular el estado externo deseado.

6. Registre la evidencia de actualización y evite la repetición ciega.

Aviso:

No vuelvas a intentar el mismo `PATCH` a ciegas. Almacene la revisión anterior, la nueva revisión, el cuerpo de validación, el hash del cuerpo del parche, el estado de la respuesta, la aprobación del revisor y la decisión de reintento fuera de BPM.

La actualización del perfil se aplica solo después de verificaciones de validación y revisión, con evidencia de auditoría explícita de éxito o conflicto.

Exporte el perfil o entregue metadatos de cumplimiento solo después de que el perfil de lectura coincida con el estado deseado aprobado.

Ejecutar transferencia de metadatos de importación, revisión, exportación y cumplimiento

Conecte un producto de cumplimiento o revisión externo para importar y exportar BPM API sin tratar los metadatos de BPM como contenido de implementación de Firefox.

Configure `$BPM_BASE_URL` y prepare un documento fuente revisado de Firefox `policies.json` más metadatos de cumplimiento externos, como escáner, ticket, revisor y decisión.

Este flujo de trabajo utiliza API-FF-001, API-PROFILE-003, API-VAL-001 y API-FF-002. BPM almacena metadatos de cumplimiento opacos en el perfil, pero el JSON de Firefox exportado solo contiene `policies` implementable.

1. Importe el documento fuente revisado a un perfil BPM.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` con `JSON {"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}`. Ejemplo de respuesta: `201 ProfileRead`.

2. Vuelva a leer el perfil creado y separe los campos BPM de la evidencia externa.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/profiles/42`. Ejemplo de respuesta: `200 ProfileRead` con `revision`, `flags` y `opaco compliance`. Ejemplos de error: `404` o `422`.

3. Revise el conjunto de políticas normalizadas antes de exportar.

El revisor verifica las claves importadas, el canal de esquema, el hash del documento fuente, los metadatos de cumplimiento y cualquier advertencia de normalización de BPM. BPM no certifica el resultado del escáner externo ni la aplicación del tiempo de ejecución de Firefox.

4. Valide el documento de implementación que se exportará.

Ejemplo de solicitud: `POST $BPM_BASE_URL/api/validate/release-153` con `JSON {"document":{"policies":{"DisableTelemetry":true}}}`. Ejemplo de respuesta: `200 {"ok":true}`. Ejemplos de error: `400`, `404`, `422` o `503`.

5. Exportar el documento de políticas canónicas de Firefox.

Ejemplo de solicitud: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Ejemplo de respuesta: `200 application/json` con `Content-Disposition` y `policies` de nivel superior. Los metadatos de cumplimiento no se exportan.

6. Escriba el registro de transferencia antes de la implementación posterior.

Aviso:

Almacenar hash de origen, perfil ID, revisión, respuesta de validación, hash de exportación, resultado del escáner, revisor y propietario posterior fuera de BPM. No trate la exportación de BPM como prueba de que Firefox ha aplicado la política.

El producto externo recibe un documento de implementación canónico de Firefox y un registro de transferencia de cumplimiento independiente con límites de propiedad claros.

Utilice herramientas de implementación fuera de BPM para colocar el `policies.json` exportado en sistemas Firefox administrados.

Iniciar el producto de control tras comprobar la integridad y la preparación de BPM

Utilice las sondas de estado actuales como puerta de inicio antes de que un producto de control externo comience la sincronización API.

La implementación del origen BPM ya está iniciada. Configure el producto externo con `$BPM_BASE_URL`, tiempo de espera, intervalo de sondeo, espera máxima y la acción a tomar cuando falla la puerta.

Este flujo de trabajo utiliza API-HEALTH-001 y API-HEALTH-002. Las sondas son sólo pequeñas señales de verificación previa; no exponen autenticación, autorización, diagnóstico de base de datos, diagnóstico de caché de esquema, solicitud identificadores ni objetivos de nivel de servicio. Este conjunto de manuales operativos nombra el inicio controlado por el estado, la propiedad de los datos, la recuperación de fallas y la regla para no volver a intentarlo a ciegas cuando la evidencia está obsoleta.

La disponibilidad del asistente de documentación es independiente de la integridad y la preparación del núcleo BPM. En 0.9.3, cada pregunta al asistente recibe el aviso publicado sobre el entrenamiento del modelo local; no se inicia ningún modelo, búsqueda, índice RAG ni solicitud a fuentes externas. Este límite no bloquea la búsqueda determinista de documentación y no debe ser una puerta para cambios de perfiles ni sincronización de API.

1. Verifique la vida básica antes de abrir la cola de sincronización.

Ejemplo de solicitud: `GET $BPM_BASE_URL/health`. Ejemplo de respuesta: `200 {"status": "ok"}`. Los errores de transporte los maneja el producto externo porque no se declara ningún cuerpo de error de aplicación BPM para esta sonda.

2. Verifique la preparación antes de las llamadas de perfil o validación.

Ejemplo de solicitud: `GET $BPM_BASE_URL/health/ready`. Ejemplo de respuesta: `200 {"status": "ready", "ready": true}`. Si `ready` no es `true`, mantenga la cola y no inicie mutaciones.

3. Registre la decisión de la puerta de inicio.

Marca de tiempo del almacén, etiqueta de URL base, tiempo de espera, estado de la sonda, cuerpo observado, versión del producto de control y siguiente acción elegida. No almacene hosts privados en ejemplos de manuales operativos compartidos.

4. Comience con inventario de solo lectura después de que pase la puerta.

Llame a `GET $BPM_BASE_URL/api/profiles?limit=50` antes de realizar operaciones de importación, exportación, validación o parcheo. Esto mantiene la sincronización inicial observable y recuperable.

5. Pasar a flujos de trabajo de mutación solo después de que las llamadas de solo lectura se realicen correctamente.

Continúe validando antes de aplicar, importando-revisando-exportando o exportando flujos de trabajo de acuerdo con el cronograma del producto externo. Mantenga la política de reintentos y retrocesos fuera de BPM.

6. Fallo cerrado cuando la puerta es ambigua.

Aviso:

Una sonda exitosa no reserva revisiones ni garantiza el éxito posterior de la API. Una sonda fallida o ambigua debe detener las mutaciones automatizadas hasta que un operador verifique los registros, el estado de implementación del origen y los límites de la red/CORS.

El producto de control externo inicia la sincronización solo cuando BPM responde al protocolo de enlace de estado y preparación actual.

Si falla la puerta de inicio, utilice los temas de resolución de problemas e implementación de origen del Administrador/DevOps en lugar de agregar suposiciones de sondeo no documentadas.

Recuperarse de fallas de integración y registrar evidencia de auditoría

Maneje fallas de validación, importación/exportación, preparación y revisión sin reintentos inseguros ni suposiciones de transacciones ocultas.

Utilice este flujo de trabajo cuando un producto externo reciba una respuesta fallida de la API de BPM, tiempo de espera, cuerpo de respuesta inesperado, `expected_revision` obsoleto o evidencia de exportación no coincidente.

Este flujo de trabajo conecta API-HEALTH-001, API-HEALTH-002, API-PROFILE-003, API-PROFILE-005, API-FF-001, API-FF-002 y API-VAL-001. Define evidencia del operador; BPM no proporciona seguimiento distribuido, reversión automática, recuperación de transacciones masivas ni notificaciones de webhooks.

1. Clasifique el error antes de volver a intentarlo.

Registre si el error es de estado/preparación, validación, importación, exportación, lectura, parche, conflicto, tiempo de espera o transporte. Conservar el estado de HTTP como 400, 404, 409, 415, 422 o 503.

2. Congela el estado externo deseado y la observación de BPM.

Método de solicitud de almacenamiento, ruta, hash de carga útil saneado, perfil ID, `revision` observado, canal de esquema, cuerpo de respuesta, tiempo de respuesta y ejecución del producto de control ID. Nunca incluya secretos ni nombres de host privados en pruebas compartidas.

3. Vuelva a verificar el estado de BPM y vuelva a leer el perfil cuando la falla sea recuperable.

Ejemplos de solicitud: `GET $BPM_BASE_URL/health`, `GET $BPM_BASE_URL/health/ready` y `GET $BPM_BASE_URL/api/profiles/42`. Si la revisión del perfil cambió, regrese a comparación y aprobación.

4. Validar o exportar nuevamente solo después de una decisión aprobada por el operador.

Para validación, repita `POST $BPM_BASE_URL/api/validate/release-153` con el mismo documento candidato. Para exportar, repita `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` y compare el hash del documento exportado.

5. Detener los reintentos de mutación en caso de revisión obsoleta o evidencia ambigua.

Si `expected_revision` devuelve 409, si el hash de exportación difiere o si los errores de validación cambiaron, no continúe con la ejecución anterior. Abra una nueva ejecución externa con lectura nueva, comparación y aprobación del revisor.

6. Publicar el registro de auditoría en el sistema de registro externo.

Aviso:

BPM almacena el estado del perfil pero no almacena un registro de auditoría de integración completo. El producto externo debe registrar el propietario, la decisión, el ticket, el recuento de reintentos, el motivo de la detención, las revisiones antiguas y nuevas, los hashes y las notas del operador.

Las fallas solo se pueden recuperar mediante evidencia explícita, lecturas nuevas y decisiones de reintento aprobadas por el operador, no mediante una repetición ciega.

Después de la recuperación, vincule el registro de auditoría a la revisión del perfil exitosa o marque la ejecución de integración como detenida.

Resolver problemas y evaluar la preparación para producción

Solucionar problemas de inicio fallido y sondas inalcanzables

Diagnostica un proceso BPM ejecutado en origen que no se inicia o no responde a las sondas de estado y preparación.

Utilice este procedimiento cuando `make dev`, `uvicorn app.main:app --reload --port 8000`, `GET /health` o `GET /health/ready` falle durante la implementación de origen o el inicio de la integración.

Este diagnóstico mantiene separadas las evidencias del proceso, el entorno y la sonda. No crea una unidad `systemd`, un servicio de Windows, un proxy inverso, un punto final TLS ni una política de reinicio de producción.

1. Capture el comando fallido y el registro de primer plano.

Ejecute o copie el comando exacto, luego registre `stdout` y `stderr` desde la terminal de primer plano. Incluya `make dev`, `uvicorn app.main:app --reload --port 8000`, resumen de rastreo de Python, directorio de trabajo y revisión del código fuente de `git rev-parse --short HEAD`.

2. Verifique el host y el puerto configurados antes de cambiar el código.

Marque `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_DATABASE_URL` y `BPM_DOCUMENTATION_SITE_DIR`. Nueva ejecución enfocada: `curl -fsS http://127.0.0.1:8000/healthy` y `curl -fsS http://127.0.0.1:8000/health/ready`.

3. Separe el inicio del proceso de la accesibilidad de la sonda.

Si el proceso no se está ejecutando, corrija primero la evidencia de dependencia, entorno o migración. Si el proceso se está ejecutando pero no se puede acceder a las sondas, verifique el firewall local, la dirección de enlace, el reenvío de puerto WSL y si la solicitud se dirige a `127.0.0.1`, `0.0.0.0` o una ruta de host de Windows.

4. Preservar los datos del usuario mientras se recopilan pruebas.

Copie registros, nombres de entorno y respuestas de sonda únicamente. No edite artefactos generados, filas de bases de datos, archivos de caché de esquema ni código fuente como recuperación de rutina. No elimine `data/bpm.db` para pasar el inicio.

5. Elija la ruta de repetición y escalamiento más pequeña.

Reejecución enfocada: para humo de producto, solo si se trata de salida de documentación y solo para contratos de documentación.

Aviso:

Si las sondas siguen siendo inaccesibles después de las comprobaciones de configuración y proceso, detenga las ejecuciones de integración automatizadas y escale con comandos, registros, nombres de configuración y evidencia de sonda.

El operador puede distinguir un proceso fallido, una sonda inalcanzable y un límite de implementación sin alterar los datos del usuario.

Después de pasar las sondas, regrese al tema de inicio controlado por estado o ejemplos de API reutilizables antes de ejecutar integraciones mutantes.

Solucionar problemas de caché de esquema y errores de validación de API

Diagnostica fallas de validación de políticas de Firefox, canales de esquema desconocidos y caché de esquema no disponible sin aplicar parches a los artefactos de esquema generados.

Utilice este procedimiento cuando `POST $BPM_BASE_URL/api/validate/release-153`, `POST $BPM_BASE_URL/api/validate/beta-999` o la validación cara al usuario devuelva 400, 404, 422 o 503; un documento que no es objeto puede devolver 200 `ok=false` con `Expected object with policy mappings`. La validación de

La validación de BPM depende de los canales de esquema mantenidos y de la configuración de esquema/caché local. El diagnóstico registra si el problema es la forma de la solicitud, un perfil desconocido, datos de políticas de Firefox no válidos o un caché de esquema no disponible.

1. Registre la solicitud y respuesta de validación exacta.

Almacena el hash del cuerpo de la solicitud desinfectado, el canal del esquema, el estado de HTTP, `detail`, `message`, `error` y `issues`. Mantenga el candidato `policies.json` fuera de los registros si contiene valores de cliente.

2. Clasifique primero los errores de forma de solicitud.

Un `document` que no es objeto o un `document.policies` faltante es entrada del operador, no un defecto de caché de esquema; BPM actual puede informar esto como 200 `ok=false`. Nueva ejecución enfocada: `POST $BPM_BASE_URL/api/validate/release-153` con `{"document":{"policies":{"DisableTelemetry":true}}}`.

3. Clasificar fallas de caché y canal de esquema.

Un perfil como `beta-999` puede devolver 404; un esquema registrado no disponible puede devolver 503. Verifique `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` y `BPM_SCHEMA_HTTP_TIMEOUT` antes de cualquier reconstrucción más amplia.

4. Utilice comprobaciones de validación y actualización mantenidas.

Nueva ejecución enfocada: para cambios en la documentación de la API o pruebas del esquema del producto cuando el comportamiento de la API cambió. No edite manualmente `app/schemas/cache` ni genere esqueletos de Firefox.

5. Detener la recuperación insegura y preservar la evidencia.

Si la misma solicitud válida cambia de estado entre ejecuciones, conserve los cuerpos de las respuestas, la revisión del origen, la ruta de la caché y las variables de entorno.

Aviso:

No edite los artefactos de esquema generados ni las filas de la base de datos como recuperación de rutina; Separe los defectos del producto de los errores de carga útil del operador.

El operador tiene un error de validación clasificado con una repetición mínima y sin mutación de esquema oculta.

Después de que la validación sea exitosa, regrese a los flujos de trabajo de validación antes de aplicar o importar-revisar-exportar.

Solucionar problemas de importación y exportación de Firefox

Diagnostique fallas de importación de JSON, importación multiparte y exportación de políticas de Firefox mientras preserve la fuente y la evidencia exportada.

Utilice este procedimiento cuando POST

`$BPM_BASE_URL/api/profiles/import/firefox/policies.json` o GET

`$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` devuelva 400, 404, 409, 415 o 422.

La solución de problemas de importación/exportación separa la validez del documento de origen, la forma del campo de varias partes, los nombres de perfiles duplicados, el acceso a perfiles archivados y la exportación canónica de Firefox de las herramientas de implementación posteriores.

1. Identifique la operación y el tipo de contenido.

Registre si la solicitud fue JSON `FirefoxPoliciesJsonImportRequest`, multipart/form-data con `file=@policies.json` o exportada con `download=1`, `pretty=1` o `include_deleted=true`.

2. Valide el documento fuente de Firefox antes de volver a intentar la importación.

Reejecución enfocada: POST `$BPM_BASE_URL/api/validate/release-153` con el mismo `document.policies`. Si la validación falla, corrija el documento fuente fuera de BPM antes de realizar otra importación.

3. Verifique los errores del operador específicos de la importación.

Para 409, elija un nuevo nombre de perfil sintético o archive/restaure intencionalmente. Para 415, verifique `application/json` o los campos de carga de varias partes. Para 422, registre los errores a nivel de campo y no reescriba el almacenamiento del perfil directamente.

4. Verifique el estado específico de la exportación.

Para 404, vuelva a leer GET `$BPM_BASE_URL/api/profiles/42?include_deleted=true`. Para perfiles archivados, decida si `include_deleted=true` es apropiado. Comparar el hash del documento exportado con el hash fuente revisado; Los metadatos del perfil BPM y los metadatos de cumplimiento no se exportan.

5. Registrar condiciones de recuperación y detención.

Ejecución enfocada: para ejemplos documentados.

Aviso:

No volver a intentarlo a ciegas después de que las pruebas de importación/exportación no coincidan; almacenar hash de origen, exportar hash, perfil ID, revisión, estado de HTTP y decisión del operador.

El operador puede identificar si el error pertenece al JSON de origen, a campos de varias partes, al estado del ciclo de vida del perfil o a la evidencia de exportación.

Después de la recuperación, adjunte la evidencia al registro de auditoría del producto de control externo.

Solucionar problemas de base de datos y almacenamiento

Diagnostique problemas locales de SQLite, ruta de almacenamiento, copia de seguridad y visibilidad del perfil sin editar las filas de la base de datos directamente.

Utilice este procedimiento cuando los perfiles desaparezcan, las escrituras fallen, las pruebas de migración no sean claras, las sumas de comprobación de las copias de seguridad no coincidan o `BPM_DATABASE_URL` apunte a una ubicación inesperada.

La implementación de origen actual utiliza almacenamiento local como `sqlite+aiosqlite:///./data/bpm.db` y archivos propiedad del repositorio en `data/`, rutas de caché de esquema y rutas de sitio de documentación. No hay copia de seguridad administrada ni orquestación de restauración en BPM.

1. Confirme la URL de la base de datos activa y el directorio de trabajo.

Registre `BPM_DATABASE_URL`, directorio actual, revisión del código fuente y si el proceso se ejecuta desde el sistema de archivos Linux o `/mnt/c`. No asuma que dos proyectiles apuntan al mismo `data/bpm.db`.

2. Verifique la evidencia de API de perfil y migración antes de abrir archivos de almacenamiento.

Reejecución enfocada: `alembic upgrade head`, luego `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`. Si la API devuelve perfiles esperados, el síntoma puede ser el filtro, el ciclo de vida o el estado del navegador.

3. Verificar la evidencia de respaldo antes de la recuperación destructiva.

Verifique los registros `mkdir -p backups`, `cp data/bpm.db backups/bpm-pre-update.db` y `sha256sum backups/bpm-pre-update.db`. Mantenga las exportaciones de perfiles `policies.json` críticos como evidencia separada.

4. Evite ediciones directas de filas de la base de datos.

No edite SQLite filas, artefactos generados o archivos de caché de esquema como recuperación de rutina. En su lugar, utilice procedimientos documentados de archivo de perfiles, restauración, importación, exportación o actualización desde el origen.

5. Escalar con evidencia de almacenamiento cuando la recuperación no sea segura.

Incluya la URL de la base de datos, la ruta del archivo, la suma de verificación, el comando de migración, la respuesta de la API y la evidencia de copia de seguridad/exportación.

Aviso:

Si no existe una copia de seguridad previa al fallo verificada, deténgase antes de eliminar o reemplazar `data/bpm.db`; BPM no proporciona recuperación en un momento dado.

El operador tiene un diagnóstico de almacenamiento que preserva los datos del usuario y separa el estado de la base de datos de UI o síntomas del filtro.

Después de que el almacenamiento esté estable, vuelva a ejecutar el estado/preparación y el flujo de trabajo de API más pequeño afectado.

Solucionar problemas de redes WSL y dependencias obsoletas

Diagnostique Windows 10/11 mediante problemas de acceso a WSL, ubicación del sistema de archivos de Linux y dependencias obsoletas de Python después de cambios en el código fuente.

Utilice este procedimiento cuando Windows no pueda acceder a un proceso BPM alojado en WSL, las sondas funcionen dentro de WSL pero no desde Windows, el rendimiento del archivo sea inestable en `/mnt/c` o las dependencias parezcan obsoletas después de `git checkout <approved-new-ref>`.

La solución de problemas de WSL mantiene separados el acceso al navegador de Windows, el enlace de procesos de Linux, la ubicación del sistema de archivos y la actualización del entorno Python. BPM no proporciona un servicio o instalador nativo de Windows en 0.9.1.

1. Confirme dónde se encuentran el entorno virtual y el pago de origen.

Prefiere el sistema de archivos Linux para el pago y `.venv`. Registre si los archivos están bajo `/mnt/c`, lo que puede cambiar el rendimiento y los permisos. No mezcle varios controles durante el diagnóstico.

2. Verifique el enlace del proceso WSL y la ruta de Windows.

Marque `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `curl -fsS http://127.0.0.1:8000/health` dentro de WSL y el acceso al navegador desde Windows. Si Windows falla, registre evidencia de firewall y reenvío de puertos.

3. Actualizar las dependencias después del movimiento de origen o cambios de revisión.

Reejecución enfocada: `source .venv/bin/activate`, `python -m pip install --upgrade pip` y `pip install -e ".[dev]"`. Si los errores de dependencia persisten, registre los nombres de los paquetes y la revisión de la fuente en lugar de editar los paquetes instalados manualmente.

4. Separe los síntomas de dependencia obsoleta de los problemas de esquema o base de datos.

Si la importación/exportación o la validación fallan después de la actualización de la dependencia, utilice el esquema/caché y la solución de problemas de importación/exportación. Si la visibilidad del perfil cambia, utilice la solución de problemas de base de datos/almacenamiento.

5. Detener suposiciones inseguras de Windows nativo.

No cree servicios de Windows no documentados, no mueva la base de datos entre sistemas de archivos ni reclame el comportamiento del instalador nativo.

Aviso:

Si la red WSL sigue siendo ambigua, detenga las integraciones externas y escale con la distribución WSL, la ruta de pago, la dirección de enlace, la URL de Windows y las salidas de la sonda.

El operador puede distinguir el enrutamiento WSL, la ubicación del sistema de archivos y los defectos de actualización de dependencias sin cambiar el almacenamiento del producto.

Después de que el acceso a WSL sea estable, regrese a la verificación de la implementación de origen y al inicio controlado por el estado.

Solucionar un portal de documentación no disponible

Recopile el estado visible de /help/ y los datos de despliegue cuando el portal de documentación instalado no esté disponible.

Use este procedimiento cuando /help/ informe de un estado ausente, obsoleto, incompatible, incompleto o no encontrado.

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

1. Registrar la URL del portal y el estado visible

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

2. Confirmar la instancia de BPM en ejecución y la configuración de documentación

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

3. Detener las acciones que dependen de la documentación

Aviso:

No considere que un portal ausente o incompatible sea un sistema de ayuda operativo ni edite manualmente los archivos instalados.

4. Escalar las pruebas al responsable del despliegue de BPM

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

5. Verificar el portal tras la recuperación

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

El portal de documentación forma parte de la interfaz de BPM. Su disponibilidad depende de que el artefacto instalado coincida con la versión de BPM en ejecución.

Verificar el portal tras la recuperación.

Revisar las preguntas sobre exposición de la red y preparación de proxy

Prepare notas de exposición de la red y preguntas sobre proxy inverso sin publicar TLS o recetas de proxy no compatibles como comportamiento de BPM.

La evaluación actual de preparación de la fuente de un solo nodo está completa y el operador sabe si los usuarios alcanzarán BPM a través de loopback, una dirección de reenvío WSL, un host de laboratorio o un proxy de propiedad externa.

BPM se puede ejecutar en un host y puerto configurados, pero la documentación de BPM no proporciona un proxy inverso oficial. TLS terminación, puerta de enlace de autenticación, política de encabezado o receta de limitación de velocidad.

1. Elija la exposición de red más pequeña que satisfaga la evaluación actual.

Prefiere 127.0.0.1. Utilice `BPM_HOST="0.0.0.0"` solo cuando el reenvío WSL o una red de laboratorio controlada lo requiera, y registre por qué se puede acceder a `BPM_PORT="8000"`.

2. Escriba las preguntas sobre la preparación del proxy antes de agregar cualquier proxy externo.

Responda quién posee los certificados TLS, el enrutamiento del nombre de host, los límites de tamaño de las solicitudes, los valores de tiempo de espera, los orígenes de CORS a través de `BPM_CORS_ALLOW_ORIGINS`, los encabezados reenviados, los registros de acceso y la respuesta a incidentes. Estas preguntas se pueden preparar ahora; Las recetas oficiales de TLS/proxy están planificadas pero aún no implementadas.

3. Verifique que las sondas aún funcionen a través del límite elegido.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Si se utiliza un proxy de propiedad externa en un laboratorio, registre tanto las sondas de origen directo como las sondas del lado del proxy. No trate una respuesta de proxy 200 como prueba de autenticación, autorización o HA de BPM.

4. Marcar explícitamente las reclamaciones de red no admitidas.

Aviso:

BPM no incluye servicios empaquetados, recetas oficiales de TLS/proxy, agrupación en clústeres de HA admitida, reglas de WAF, firma de solicitudes, limitación de velocidad, SSO, OAuth/OIDC, control de acceso basado en roles ni refuerzo de producción. Manténgalos como controles externos o requisitos diferidos.

La nota de red separa lo que se puede configurar ahora del proxy inverso y las decisiones TLS que permanecen externas o diferidas.

Utilice el tema de la ventana de supervisión, copia de seguridad y actualización antes de que comience una ventana de evaluación compartida.

Planificar entradas de monitoreo, evidencia de respaldo y ventanas de actualización

Prepare evidencia operativa del estado actual para una instancia ejecutada en origen y, al mismo tiempo, evite reclamos sobre monitoreo administrado, automatización de restauración oficial o actualizaciones continuas.

La instancia pasó la revisión de preparación del código fuente y exposición de la red, y un único administrador o propietario de DevOps es responsable de la ventana de evaluación.

Hasta que existan decisiones sobre distribuciones y servicios de tiempo de ejecución, la planificación de operaciones debe centrarse en entradas observables, copias de seguridad manuales y evidencia de exportación, y ventanas de actualización explícitas para un nodo de origen.

1. Enumere las entradas de monitoreo que existen hoy.

Estado del proceso de registro, salida del terminal en primer plano, `stdout`, `stderr`, `GET /health`, `GET /health/ready`, resultados de ejemplo de API seleccionados, acceso a la documentación a través de `/help/` y la revisión de la fuente de `git rev-parse --short HEAD`.

2. Recopile copias de seguridad y exporte evidencia antes de la ventana.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Exporte también documentos representativos de Firefox como `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` y almacene la evidencia resultante `policies.json` con sumas de verificación.

3. Programe una ventana de actualización que detenga el mundo para cambios de fuente.

Para 0.9.1, planifique el tiempo de inactividad de un nodo antes de `git fetch --tags`, actualización de dependencia, `alembic upgrade head`, y . Se planean actualizaciones continuas, pero aún no se han implementado.

4. Registre las lagunas como obstáculos, no como suposiciones ocultas.

Aviso:

BPM no proporciona monitoreo administrado, programación de respaldo automatizada, automatización de restauración oficial, recuperación en un momento dado, secretos administrados, rotación de tokens ni orquestación de actualizaciones continuas. Utilice pruebas manuales y condiciones de detención del runbook de actualización.

El plan de operación contiene entradas de monitoreo, evidencia de respaldo/exportación y una ventana de actualización realista para la operación de origen actual.

Adjunte esta evidencia al HA y al registro de límites diferidos para que el trabajo de distribución posterior pueda reutilizar la misma lista de verificación.

Asistente de documentación de BPM

Operar el asistente de documentación en BPM 0.9.3

Verifique el límite publicado del aviso de entrenamiento sin hacer que BPM ni la búsqueda determinista dependan del chat.

Use un checkout de BPM 0.9.3 compatible y los recursos base de [Requisitos mínimos del sistema para BPM](#).

En esta versión, el chat es solo una interfaz localizada para el lector. No inicia ni instala un modelo, no recupera documentación, no usa un índice E5, no genera una respuesta, no muestra citas ni contacta fuentes externas. La búsqueda determinista de documentación sigue siendo independiente y está disponible.

1. Inicie BPM con el flujo normal de código fuente compatible.

No añada configuración de modelo, índice, embeddings ni proveedor externo al inicio de BPM 0.9.3.

2. Abra el portal de documentación en el idioma del lector.

Compruebe que el asistente se abre desde la esquina inferior derecha y que los controles de búsqueda ordinarios no cambian.

3. Envíe una pregunta sobre BPM.

La transcripción debe mostrar el aviso localizado de que el modelo local se está entrenando y la función estará disponible en una versión posterior.

4. Envíe una pregunta ajena al producto.

Se espera el mismo aviso; el chat no es un asistente de uso general en esta versión.

5. Busque la respuesta actual mediante la búsqueda determinista de documentación.

La búsqueda, los filtros, la navegación y el orden de resultados siguen siendo la vía compatible para la información del producto.

El asistente muestra el aviso de entrenamiento localizado publicado, mientras BPM y la búsqueda determinista siguen siendo utilizables de forma independiente.

No instale un modelo ni cree una generación RAG para 0.9.3. Registre como defecto de documentación cualquier diferencia en el aviso, la configuración regional o el límite con la búsqueda ordinaria.

Mantener el límite del asistente de documentación en BPM 0.9.3

Mantenga correcto el aviso de entrenamiento localizado y el límite con la búsqueda determinista al actualizar la documentación de BPM.

Tenga preparada la revisión de código fuente BPM aprobada y el flujo de publicación de documentación.

En 0.9.3 no hay un modelo local, artefacto de modelo, generación RAG, almacén de embeddings, almacén de citas ni proveedor de fuentes externas que mantener. No cree estado de mantenimiento para funciones que esta versión no entrega.

1. Aplique la revisión BPM aprobada y vuelva a compilar la documentación.

Use el flujo documental normal, incluido `make docs-install-dev` al preparar un entorno de desarrollo.

2. Abra la documentación publicada en todos los idiomas compatibles.

Compruebe la entrada del asistente y el funcionamiento independiente de la búsqueda determinista.

3. Envíe una pregunta en cada configuración regional.

Compruebe el aviso de cada idioma y la ausencia de respuesta, fuente, cita, afirmación externa o instalación de modelo.

4. No ejecute comandos de instalación de modelo ni de generación RAG para 0.9.3.

Pertenecen a una versión posterior y no recuperan el aviso de entrenamiento.

5. Registre la revisión de documentación y el resultado de la verificación.

Trate cualquier diferencia entre el aviso publicado y el límite de búsqueda determinista como un defecto de documentación.

El portal publicado conserva un aviso localizado en el chat y una ruta independiente hacia la información actual mediante la búsqueda determinista.

No conserve preguntas de lectores como artefacto de mantenimiento. Reevalúe las operaciones de modelo y recuperación solo cuando una versión posterior las entregue de forma explícita.